Lecture4 数据智能计算架构设计



于晗

han_yu@sjtu.edu.cn

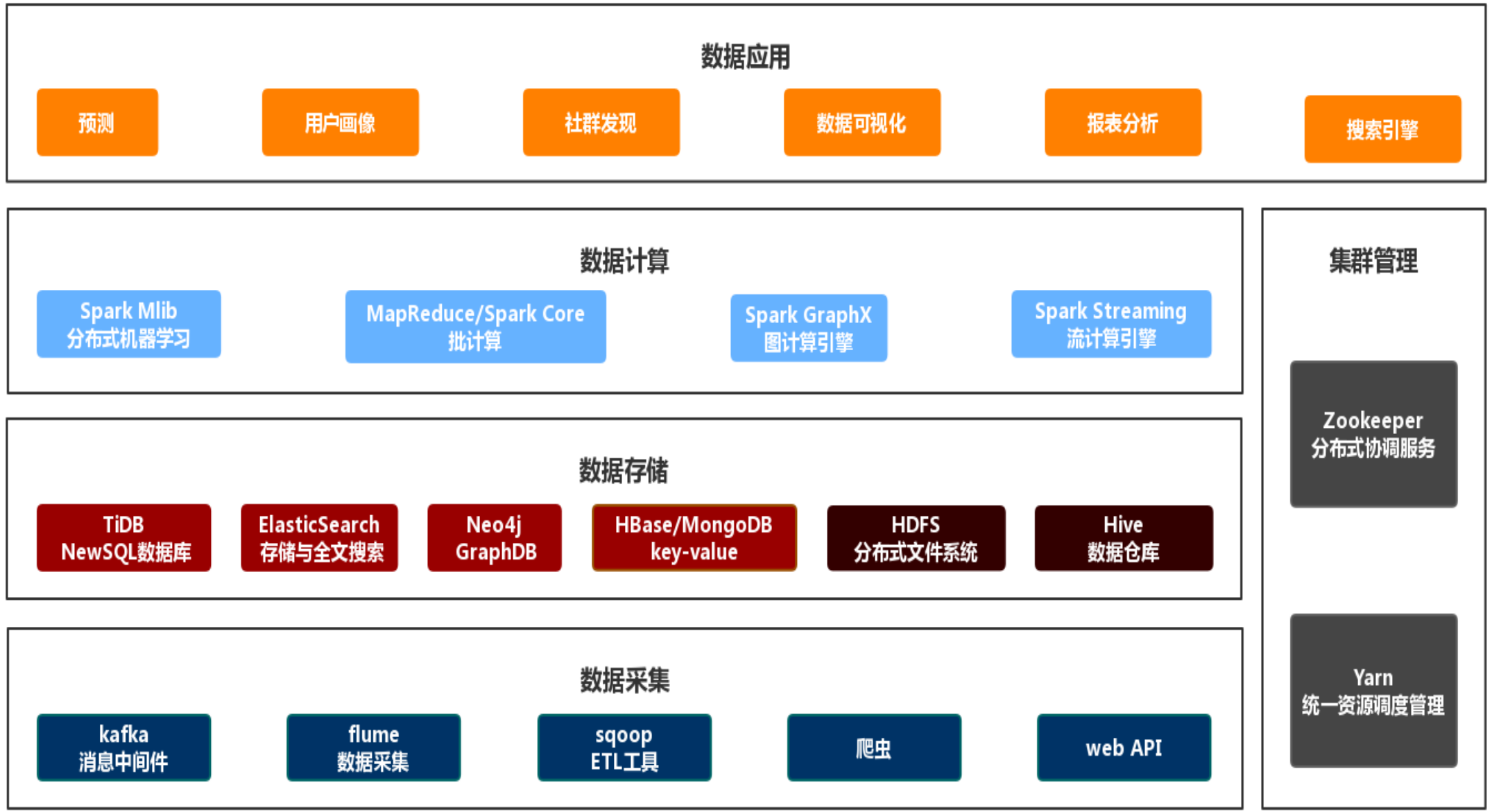
目录

- 1 数据智能平台计算架构回顾
- 2 流计算模式设计
- 3 图计算模式设计
- 4 小结

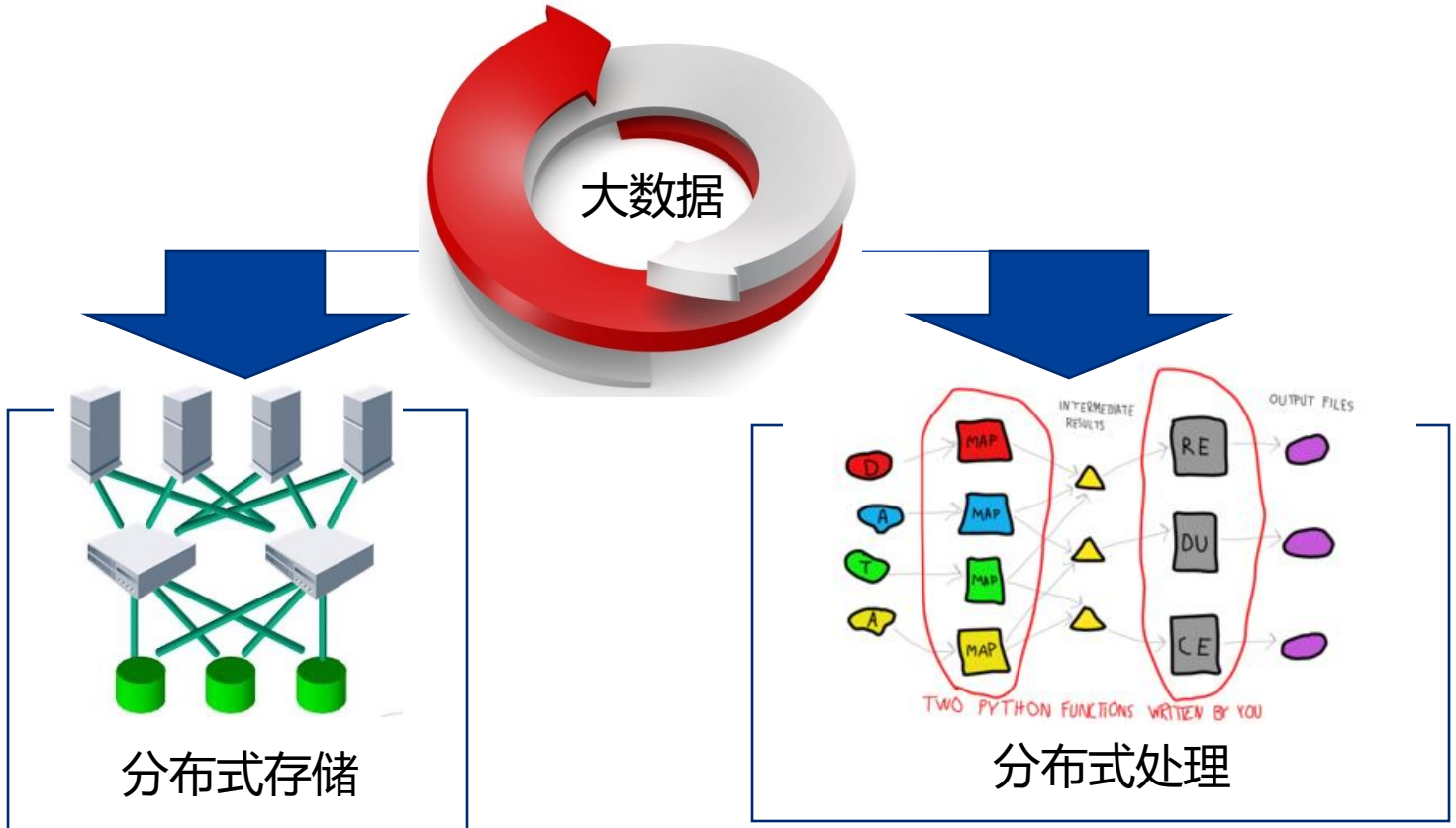
数据分析的技术框架

技术层面	功能
数据采集	利用 ETL 工具将分布的、异构数据源中的数据如关系数据、数据文件等，抽取到临时中间层后进行清洗、转换、集成，最后加载到数据仓库或数据集市中，成为联机分析处理、数据挖掘的基础； 或者也可以把实时采集的数据作为流计算系统的输入，进行实时处理分析
数据治理和存储	利用分布式文件系统、数据仓库、关系数据库、NoSQL 数据库、云数据库等，实现对结构化、半结构化和非结构化海量数据的存储和管理
数据查询与分析应用	利用分布式并行编程模型和计算框架，结合机器学习和数据挖掘算法，实现对海量数据的处理和分析；对分析结果进行可视化呈现，帮助人们更好地理解数据、分析数据
数据隐私和安全	在从大数据中挖掘潜在的巨大商业价值和学术价值的同时，构建隐私数据保护体系和数据安全体系，有效保护个人隐私和数据安全

一个典型的大数据平台架构



大数据分析的两大核心技术



GFS\HDFS
BigTable\HBase
NoSQL（键值、列族、图形、文档数据库）
NewSQL（如：SQL Azure）

MapReduce

大数据计算模式

大数据计算模式	解决问题	代表产品
Velocity **流计算**	针对流数据的实时计算	Flink、Storm、S4、Flume、Streaming、Puma、DStream、Super Mario、银河流数据处理平台等
Value **图计算**	针对大规模图结构数据的处理	Neo4J，Pregel、GraphX、Giraph、PowerGraph、Hama、GoldenOrb 等
Variety **存储计算**	大规模数据的存储管理和查询分析	Dremel、Hive、Cassandra、Impala 等
Volume **批处理计算**	针对大规模数据的批量处理	MapReduce、Spark 等

目录

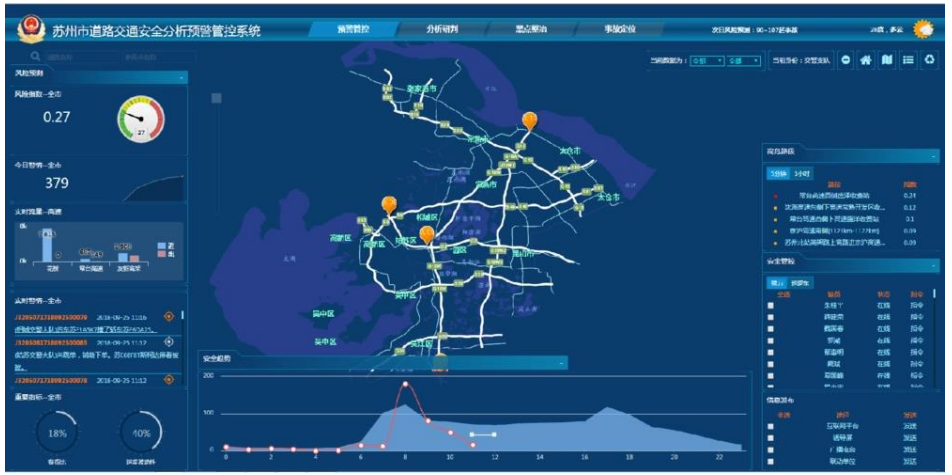
- 1 数据智能平台计算架构回顾
- 2 流计算模式设计
- 3 图计算模式设计
- 4 小结

2. 流计算模式

- 近年来，在 Web 应用、网络监控、传感监测等领域，兴起了一种新的数据密集型应用——流数据，即数据以大量、快速、时变的流形式持续到达。
- 实例：PM2.5检测、电子商务网站用户点击流

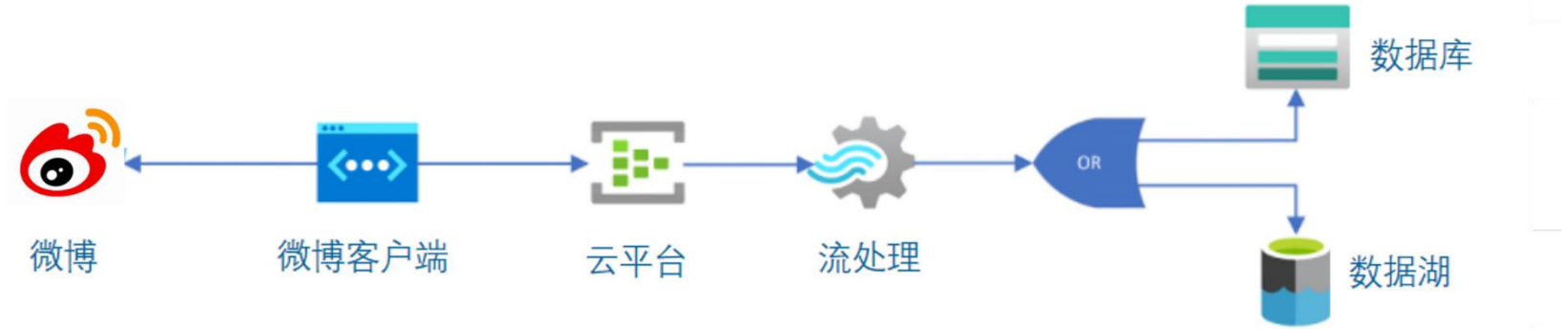






流计算应用场景

- 新浪微博：WAIC实时流计算平台
- 随着新浪微博业务的不断推进，对数据处理的实时性要求越来越高。例如，大家所熟悉的微博热词热搜，需要在很短的时间内完成数据处理以供在线系统使用。
- WAIC 实时流计算平台为新浪微博提供可靠的毫秒级和秒级实时数据处理服务，通过提供统一的数据源和配置化接入方式，帮助提高新浪微博实时作业的开发效率，降低部门开发与运营的成本。



流计算应用场景

- 物联网：传感器数据异常检测
- 随着物联网技术的不断升级，物联网中各类设备的种类、数量不断增加，物联网对设备的实时异常检测要求越来越高。
- 警报指标，满足物联网实时监控和预警借助流计算框架，能实时检测传感器数据是否符合的需求。

```
t=keyedTable(`deviceID,100:0,`deviceID`time,[INT,DATETIME])
deviceNum=3
insert into t values(1..deviceNum,take(now().datetime(),deviceNum))
def checkNoData (mutable keyedTable, mutable outputTable, msg) {
        keyedTable.append!(select deviceID, ts from msg)
        warning = select now().datetime(), deviceID, 1 as anomalyType
        if(warning.size() > 0) outputTable.append!(warning)
}
```

基于DolphinDB流计算框架，实现传感器数据异常检测

time	deviceID	anomalyType	anomalyString
2020.08.20T11:42:30	1	0	sum(temperature > 40) > 2 && sum(temperature > 30) > 3
2020.08.20T11:42:30	3	0	sum(temperature > 40) > 2 && sum(temperature > 30) > 3
2020.08.20T11:43:30	2	0	sum(temperature > 40) > 2 && sum(temperature > 30) > 3
2020.08.20T11:47:26	3	1	

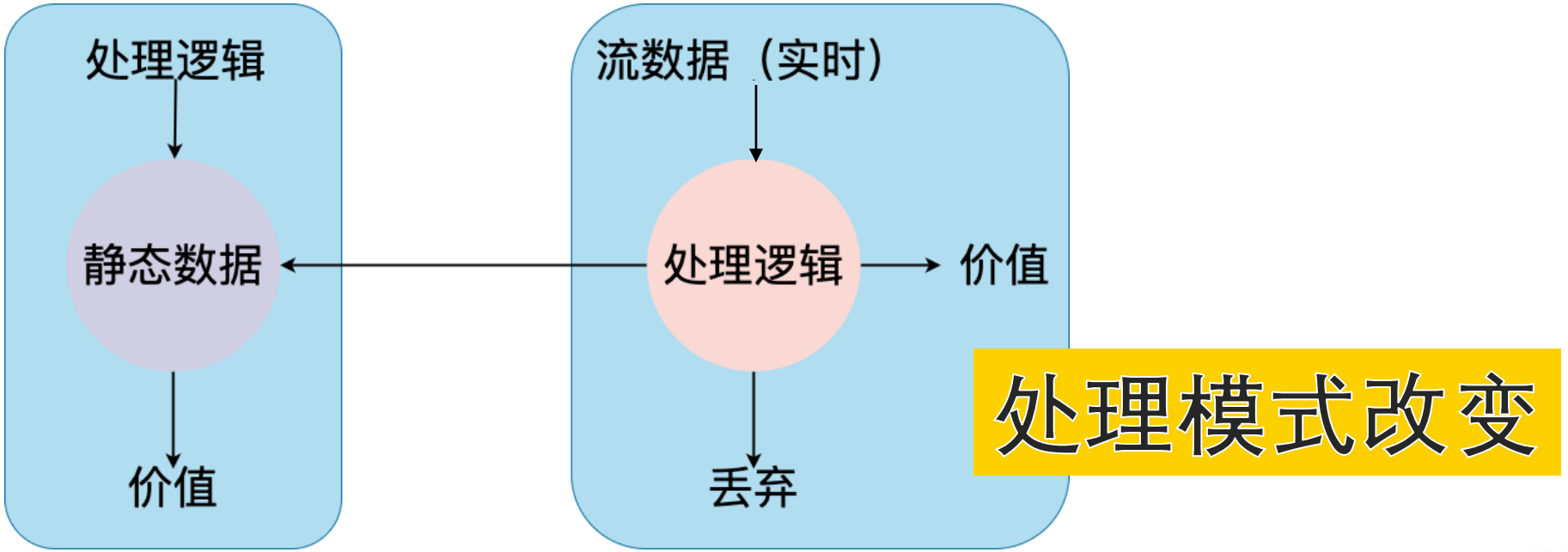
传感器异常报警信息

流数据特点

- 流数据具有如下特征：
  - 无限性：数据快速持续到达，潜在大小也许是无穷无尽的。
  - 易失性：数据量大，但是不十分关注存储，一旦经过处理，要么被丢弃，要么被归档存储。
  - 弱语义性：注重数据的整体价值，不过分关注个别数据。
  - 时序敏感：对数据的时序和完整性较为敏感。当数据顺序颠倒，或者不完整时，那么系统将无法控制新到达的数据元素的顺序，相应处理可能不准确甚至无法处理。
  - 突发性：数据的流入速率和顺序并不确定，甚至会有较大的差异。这要求系统要有较高的吞吐量，能快速处理大数据流量。

批量计算vs流计算

- 对静态数据和流数据的处理，对应着两种截然不同的计算模式：**批量计算**和**实时计算**。
  - 批量计算：**充裕时间**处理静态数据，如 Hadoop。
  - 流数据不适合采用批量计算，因为流数据不适合用**传统的关系模型建模**。（弱语义性）
  - 流数据必须采用**实时计算**，响应时间为秒级。



数据的两种处理模型图

扩展：SGD也是一种流计算算法

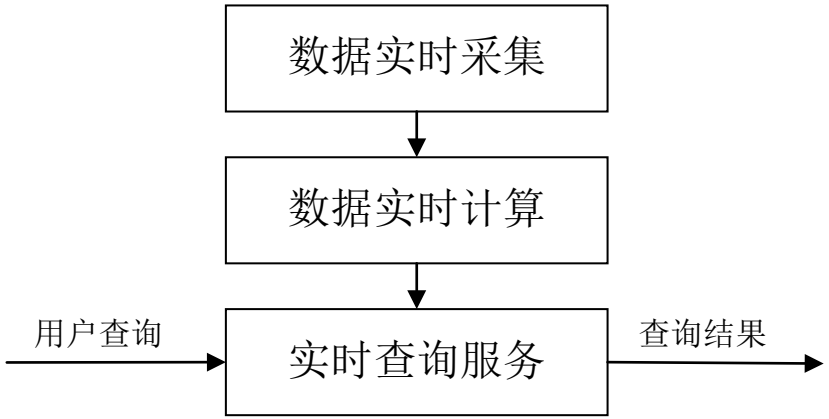
- **随机梯度下降**Stochastic Gradient Descent (SGD)
- **在机器学习中，我们称之为在线学习**
  - 问题建模的输入是一个持续数据流
  - 使用一个算法从数据流中学习并满满适应它的变化
- **思路：缓慢更新模型**
  - SGD (SVM, Perceptron) 产生小的更新
  - 首先在训练数据上训练一个分类器模型
  - 然后对于数据流中的每个样例，用小学习率缓慢地更新模型参数

流计算系统的设计需求

- 流计算秉承一个基本理念，即**数据的价值随着时间的流逝而降低**，如用户点击流。
- 因此，当事件出现时就应该立即进行处理，而不是缓存起来进行批量处理。为了及时处理流数据，就需要一个低延迟、可扩展、高可靠的处理引擎。
- 对于一个流计算系统来说，它应达到如下需求：
  - 高性能：处理大数据的基本要求，如每秒处理几十万条数据。
  - 海量式：支持TB级甚至是PB级的数据规模。
  - 实时性：保证较低的延迟时间，达到秒级别，甚至是毫秒级别。
  - 分布式：支持大数据的基本架构，必须能够平滑扩展。
  - 易用性：能够快速进行开发和部署。
  - 可靠性：能可靠地处理流数据。

2.1 流计算处理过程

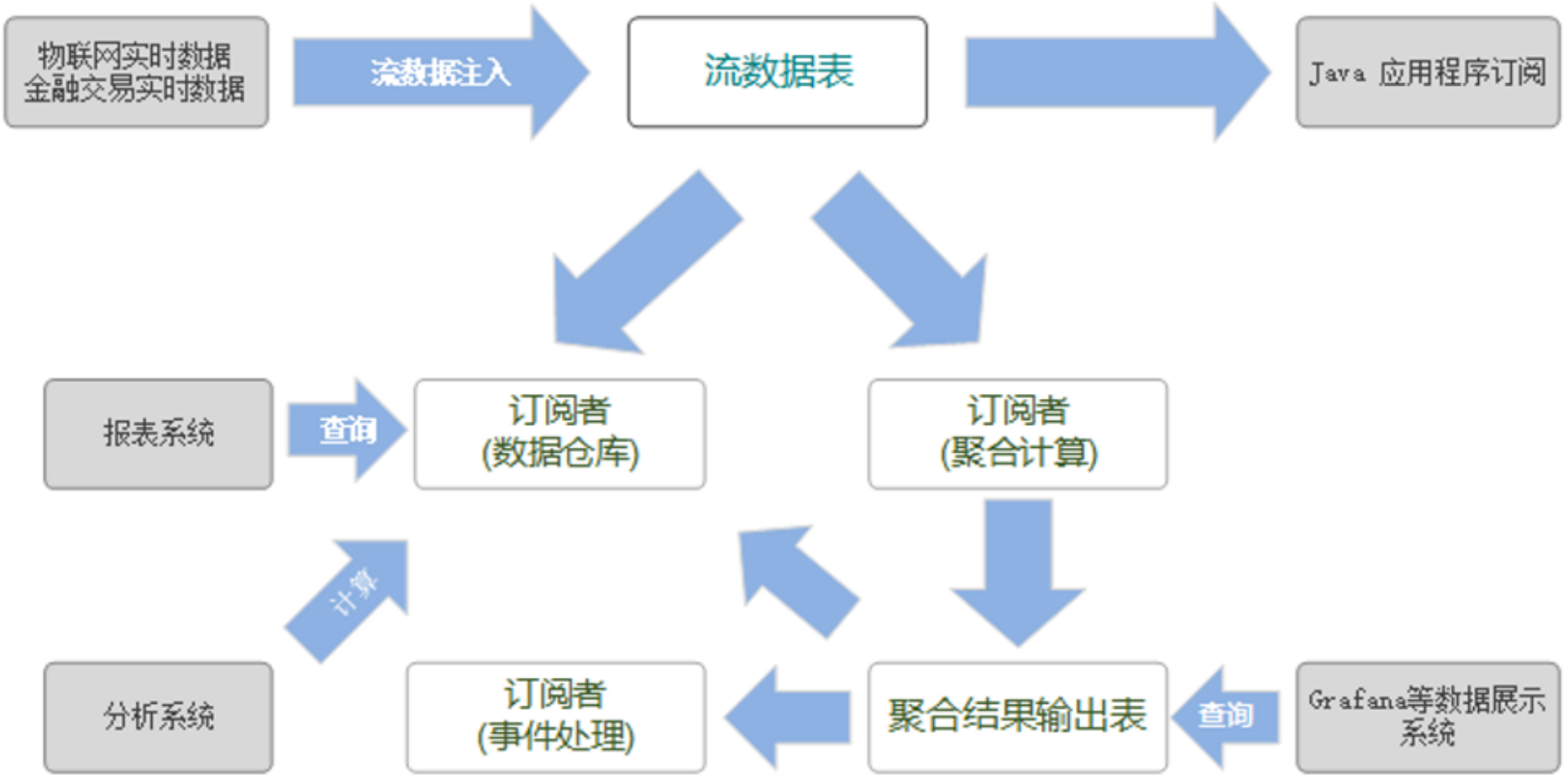
- 数据实时采集阶段通常采集多个数据源的海量数据，需要保证**实时性**、**低延迟**与**稳定可靠**。
  - Facebook 的 Scribe。
  - LinkedIn 的 Kafka。
  - 淘宝的 Time Tunnel。
  - 基于 Hadoop 的 Chukwa 和 Flume。
- 经流处理系统处理后的数据，可视情况进行存储，以便之后再进行分析计算。在时效性要求较高的场景中，处理之后的数据也可以直接丢弃。
- 传统的数据处理流程，用户需要主动发出查询才能获得想要的结果。而在流处理流程中，实时查询服务可以不断更新结果，并将用户所需的结果实时推送给用户。



2.1 流计算处理过程

- 可见，流处理系统与传统的数据处理系统有如下不同：
  - 流处理系统处理的是**实时数据**，而传统的数据处理系统处理的是预先存储好的**静态数据**。
  - 用户通过流处理系统获取的是**实时结果**，而通过传统的数据处理系统，获取的是过去**某一时刻的结果**。
  - 流处理系统无需用户主动发出查询，实时查询服务可以**主动将实时结果推送**给用户。

一个典型的流数据处理流程



流计算框架

现有流计算框架：

- 商业级：IBM InfoSphere Streams和IBM StreamBase
- 较为常见的是开源流计算框架，代表如下：
  - Twitter Storm：分布式实时计算系统，可简单、高效、可靠地处理大量的流数据
  - Apache Flink：开源流计算平台，是通用的、分布式的、可扩展的、分区容错的、可插拔的流式系统
- 公司为支持自身业务开发的流计算框架：
  - Facebook Puma
  - Dstream（百度）
  - 银河流数据处理平台（淘宝）











2.2 Storm架构

基本概念

stream：Storm将流数据stream描述成一个无限的tuple序列，这些序列会以分布式的方式并行地创建和处理。每个tuple是一堆值，每个值有一个名字，并且每个值可以是任何类型。tuple是一个value list（值列表）。




```
public Stream newStream(String txId, IRichSpout spout) {
    return newStream(txId, new RichSpoutBatchExecutor(spout));
}

public Stream newStream(String txId, IPartitionedTridentSpout spout) {
    return newStream(txId, new PartitionedTridentSpoutExecutor(spout));
}

public Stream newStream(String txId, IOpaquePartitionedTridentSpout spout) {
    return newStream(txId, new OpaquePartitionedTridentSpoutExecutor(spout));
}

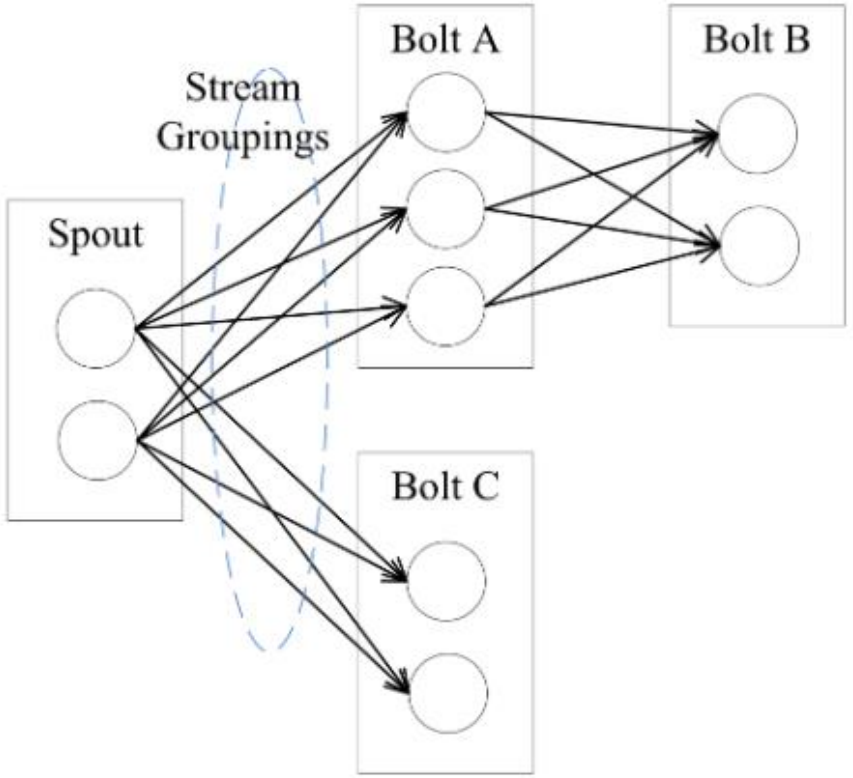
public Stream newStream(String txId, ITridentDataSource dataSource) {
    if (dataSource instanceof IBatchSpout) {
        return newStream(txId, (IBatchSpout) dataSource);
    } else if (dataSource instanceof ITridentSpout) {
        return newStream(txId, (ITridentSpout) dataSource);
    } else if (dataSource instanceof IPartitionedTridentSpout) {
        return newStream(txId, (IPartitionedTridentSpout) dataSource);
    } else if (dataSource instanceof IOpaquePartitionedTridentSpout) {
        return newStream(txId, (IOpaquePartitionedTridentSpout) dataSource);
    } else {
        throw new UnsupportedOperationException("Unsupported stream");
    }
}
```

Storm源码——新建Stream

2.2 Storm架构

- 基本概念

- spout：stream的源头。spout会从外部数据源读取数据，然后封装成tuple形式，发送到stream中。spout是一个主动的角色，在接口内部有个nextTuple函数，storm框架会不停的调用该函数。
- bolt：stream的状态转换过程抽象为bolt。bolt可以处理tuple，也可以将处理后的tuple作为新的stream发送给其他bolt。bolt可以执行过滤、函数操作、join、操作数据库等任何操作。bolt是被动的角色，接口中有一个execute方法，在接收到消息之后会调用此函数，用户可以在此方法中执行自己的处理逻辑。
- topology：每一个组件都是并行运行的。bolt中包含多个task，可以在不同机器节点并行地处理，topology中可以指定每个组件的并行度。



2.2 Storm架构

- 基本概念

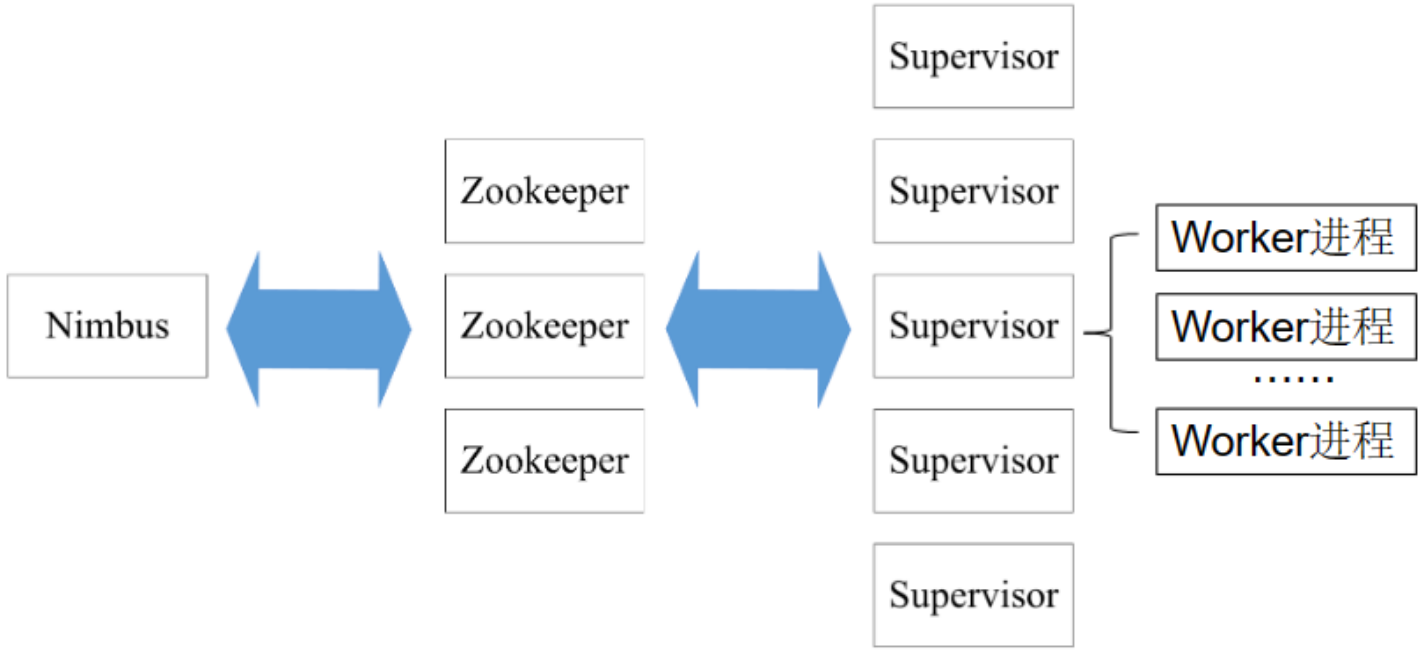
- stream groupings：stream的分组方式，定义一个流如何分配Tuple到Bolt。
- storm提供六种分组方式：NonGrouping, AllGrouping, ShuffleGrouping, DirectGrouping, GlobalGrouping, FieldsGrouping。
- 实际应用中可以根据具体需求选择分组方式，如ShuffleGrouping可以实现较好的负载均衡，而FieldsGrouping则类似于SQL group by关键字，主要用于数据分组统计。



2.2 Storm架构

Storm集群部署示意图

- 借助于Zookeeper，若Nimbus进程或Supervisor进程意外终止，重启时也能读取、恢复之前的状态并继续工作，使得Storm极其稳定

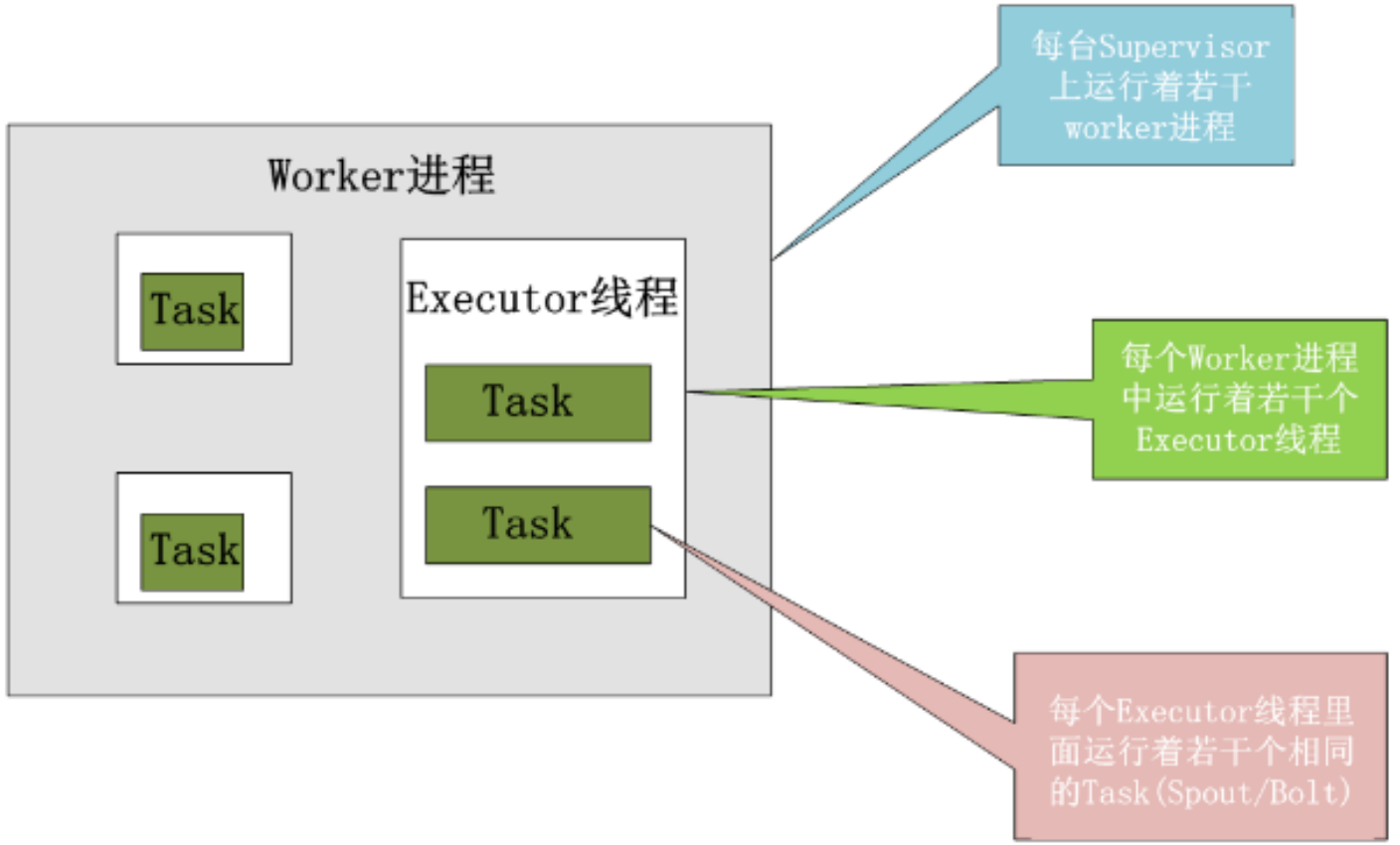


Storm集群架构示意图

2.2 Storm架构

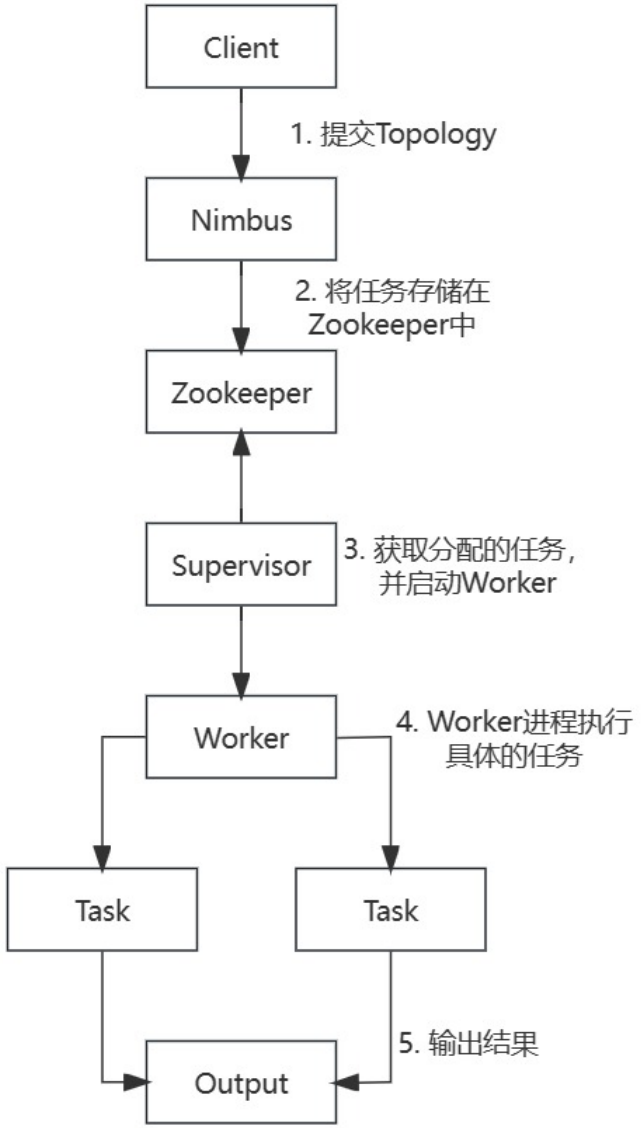
Worker进程

- 每个Worker进程都属于一个特定的Topology，每个Supervisor节点的Worker可以有多个，每个worker对应Topology中的一个组件（Spout或Bolt）运行一个或者多个executor线程来提供task的运行服务



2.2 Storm架构

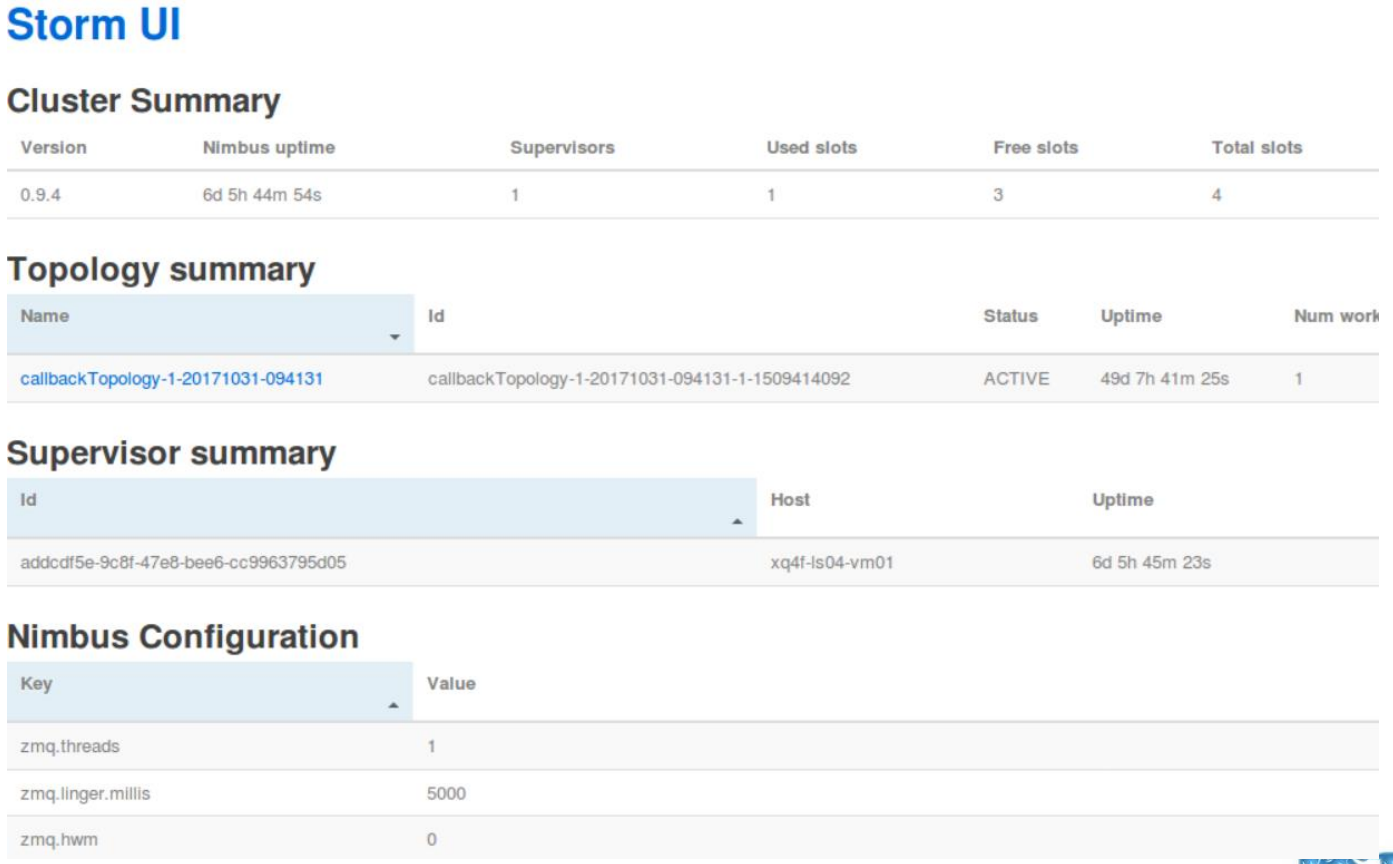
- 工作流程
- 所有的topology任务的提交必须在storm客户端节点上进行，提交后，由nimbus节点分配给其他supervisor节点进行处理。
- nimbus节点首先将提交的topology进行分片，分成一个个task，分配给相应的supervisor，并将task和supervisor相关的信息提交到zookeeper集群上。
- supervisor会去zookeeper集群上认领自己的task，通知自己的worker进程进行task的处理。



Storm工作流程示意图

2.2 Storm架构

- Storm依赖
  - 搭建Zookeeper集群
  - 安装Storm依赖库（ZeromQ,JZMQ,JDK,Python）
  - 下载并解压Storm发布版本
  - 修改storm.yaml配置文件
  - 启动Storm各个后台进程



Storm UI

Cluster Summary

Version	Nimbus uptime	Supervisors	Used slots	Free slots	Total slots
0.9.4	6d 5h 44m 54s	1	1	3	4

Topology summary

Name	Id	Status	Uptime	Num work
callbackTopology-1-20171031-094131	callbackTopology-1-20171031-094131-1-1509414092	ACTIVE	49d 7h 41m 25s	1

Supervisor summary

Id	Host	Uptime
addcdf5e-9c8f-47e8-bee6-cc9963795d05	xq4f-ls04-vm01	6d 5h 45m 23s

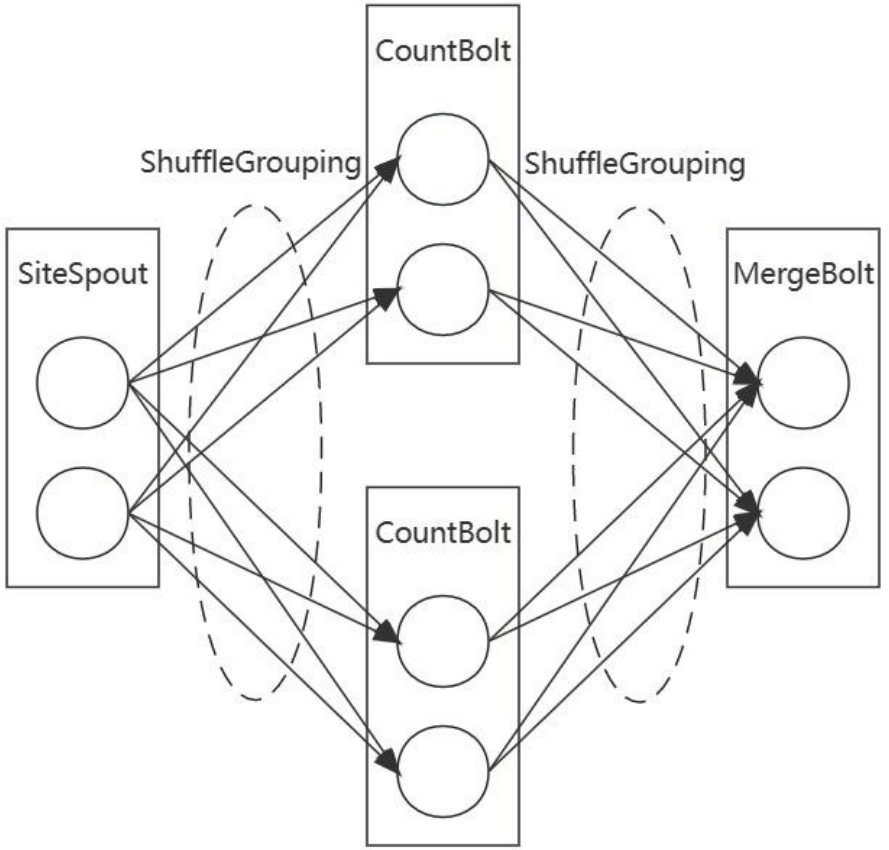
Nimbus Configuration

Key	Value
zmq.threads	1
zmq.linger.millis	5000
zmq.hwm	0

2.2 Storm架构

实例：实时分析——双十一实时监控销售额

- Spout:
  - SiteSpout: 接收实时订单消息
- Bolt:
  - CountBolt: 计算当前订单的交易金额
  - MergeBolt: 合并计算结果并写入数据库
- Stream Grouping:
  - ShuffleGrouping



2.2 Storm架构

- 实例：实时分析——双十一实时监控销售额

```
@Override
public void declareQutputFields(OutputFieldsDeclarer declarer)
    //申请输出的字段时间、金额
    declarer.declare(new Fields("time", "money"));
}
@Override
public void nextTuple(){
try
{
BytesMessage message = (BytasMessageconsumer receive(10000);
    if(null != message)
    {
        int len = (int) message-getBodyLength();
        byte[ ]data = new byte[len];
        message.readBytes(data,len);
        String line = new String(data);
        String[] arr = line.split("\\");
        //金额
        double money = Double.parseDoubLe(ark[21);
        collector.emit(new values(arr[1],money));
    }
}catch (Exception e)
```

SiteSpout

```
public void execute(Tuple input){
    double money = input-getDouble(1);
    sum += money;
}
@Override
public void prepare(Map stormConf, TopologyContext context ,OutputCollector{
    this.collector = collector;
    timer = new Timer();
    timer.schedule(this, 0L,1000L);
}
@Override
public void run(){
    currentTime =DateUtil.getSysdate();
    //每分钟输出一次
    if( "00".equals( currentTime.substring(17))){
        collector.emit(new values(sum));
    }
}
@Override
public void declareOutputFields(ututFieldsDeclarer declarer){
//输出金额
declarer.declare( new Fields( "money" ) );
```

CountSpout

2.2 Storm架构

- 实例：实时分析——双十一实时监控销售额

```
public statie void main(String[]args) {
    TopologyBuilder builder = new TopologyBuilder( );
    builder.setSpout("SiteSpout ", new SiteSpout(), 1).setMaxTaskParallelism(2);
    builder.setBolt("countBolt", new CountBolt(), 4 ).shuffleGrouping("SiteSpout");
    builder.setBolt("mergeBolt", new MergeBolt(), 1).shuffleGrouping("CountBolt");
    Config config =new Config();
    config.setDebug(false);
    config.put(Config.TOPOLOGY_MAX_SPOUT_PENDING, 1);
    config.put( "CountBolt_param_num", 4);
    try
    {
        if (args != null&& args.length > 0) {
            StormSubmitter.submitTopoLogy(args[0], config,builder.createTopology());
        } else {
            LocalCluster cluster = new Localcluster();
            cluster.submitTopology("taobao_double11_bill_topology",
                                   config, builder.createTopology());
        }
    catch (Exception e){
        e.printstackTrace();
    }
```

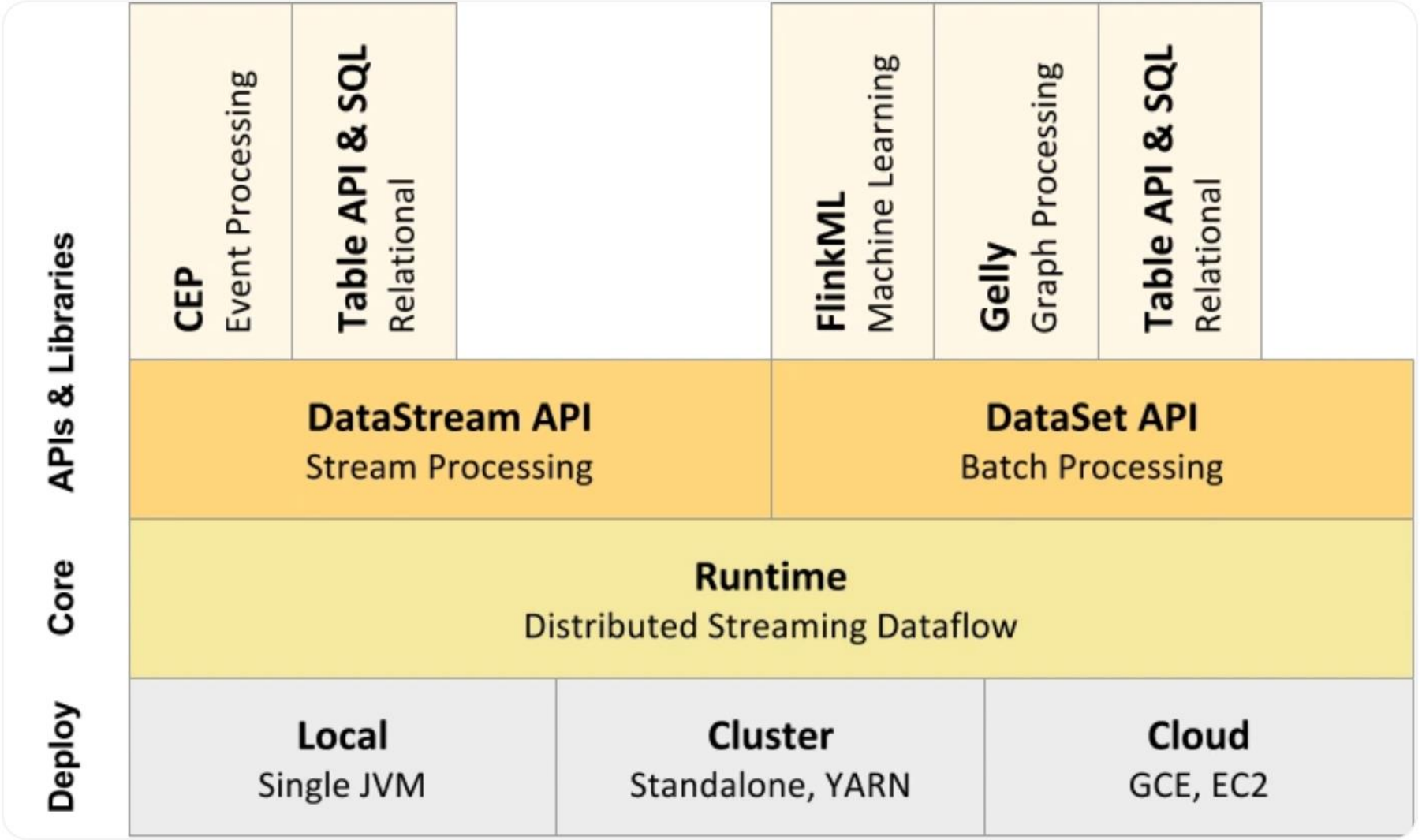
Topology

2.3 Flink架构

- Apache Flink 是一个框架和分布式处理引擎，用于在无边界和有边界数据流上进行有状态的计算。Flink 能在所有常见集群环境中运行，并能以内存速度和任意规模进行计算。
- 世界各地有很多要求严苛的流处理应用都运行在 Flink 之上。
- Flink是目前开源社区中唯一一套集高吞吐、低延迟、高性能三者于一身的分布式流式处理框架。像Apache Spark也只能兼顾高吞吐和高性能特性，主要因为在Spark Streaming流式计算中无法做到低延迟保障；而流式计算框架Apache Storm只能支持低延迟和高性能特性，但是无法满足高吞吐的要求。
- 批与流
  - 批处理的特点是有界、持久、大量，非常适合需要访问全套记录才能完成的计算工作，一般用于离线统计。
  - 流处理的特点是无界、实时，无需针对整个数据集执行操作，而是对通过系统传输的每个数据项执行操作，一般用于实时统计。

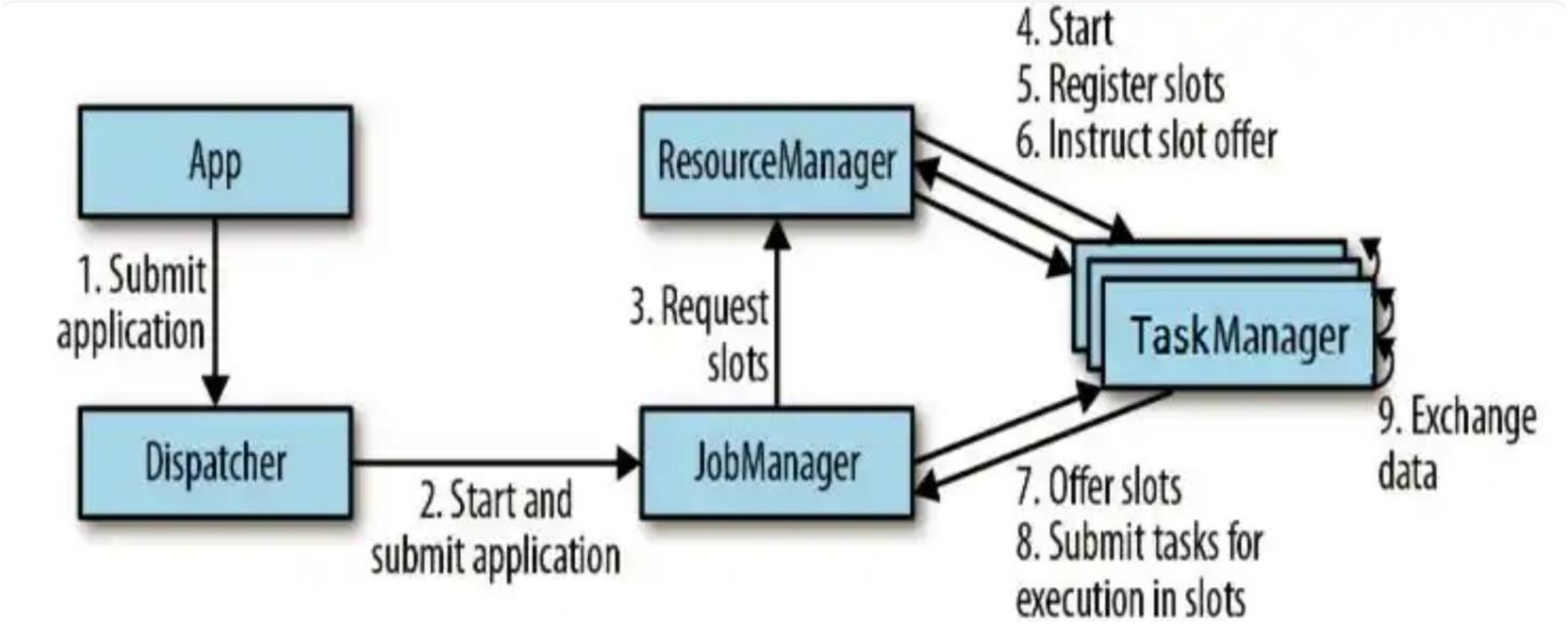
2.3 Flink架构

- 整体架构



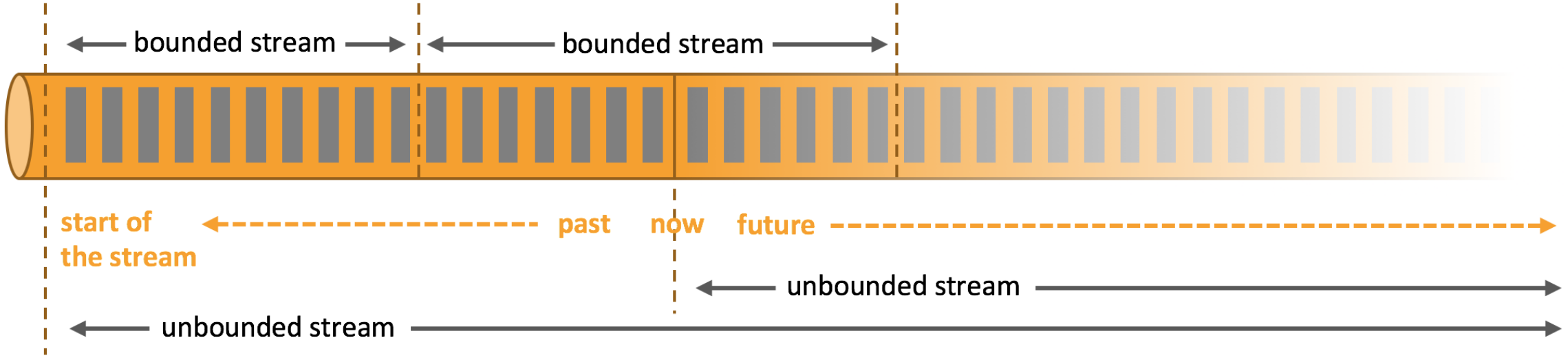
2.3 Flink架构

- 工作流程



2.3 Flink架构

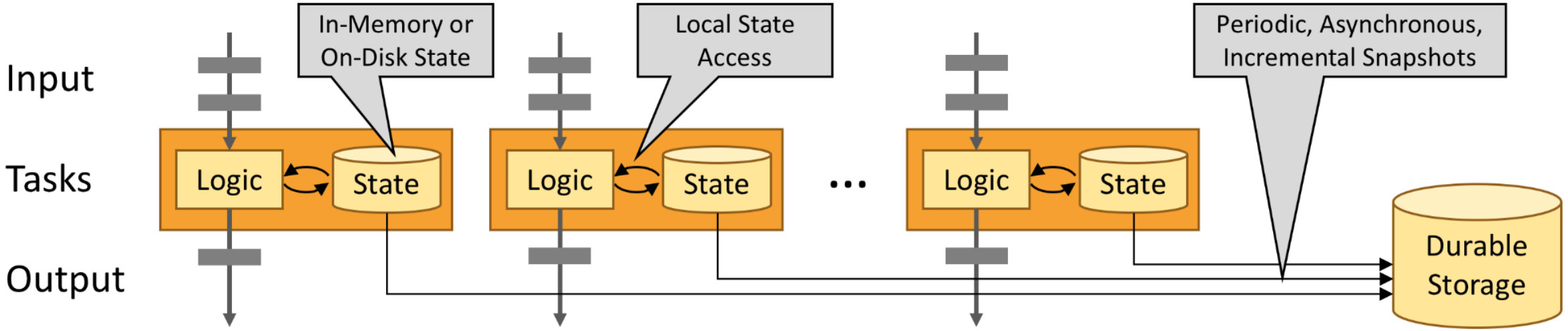
- 批流一体
- 无界流：有定义流的开始，但没有定义流的结束。它们会无休止地产生数据。无界流的数据必须持续处理，即数据被摄取后需要立刻处理。我们不能等到所有数据都到达再处理，因为输入是无限的，在任何时候输入都不会完成。处理无界数据通常要求以特定顺序摄取事件，例如事件发生的顺序，以便能够推断结果的完整性。
- 有界流：有定义流的开始，也有定义流的结束。有界流可以在摄取所有数据后再进行计算。有界流所有数据可以被排序，所以并不需要有序摄取。有界流处理通常被称为批处理。



2.3 Flink架构

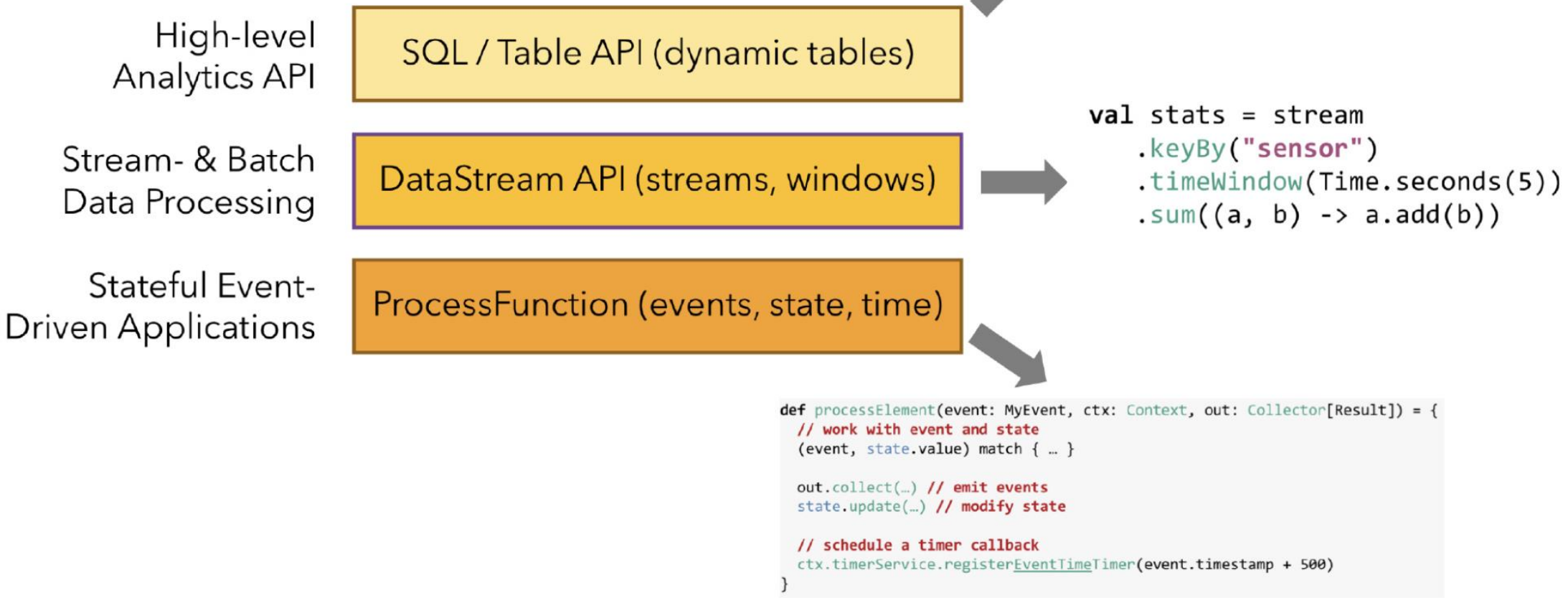
Flink支持有状态计算

- 内存+硬盘的状态存储支持实时计算
- 状态支持事务计算



2.3 Flink架构

- Flink分层API



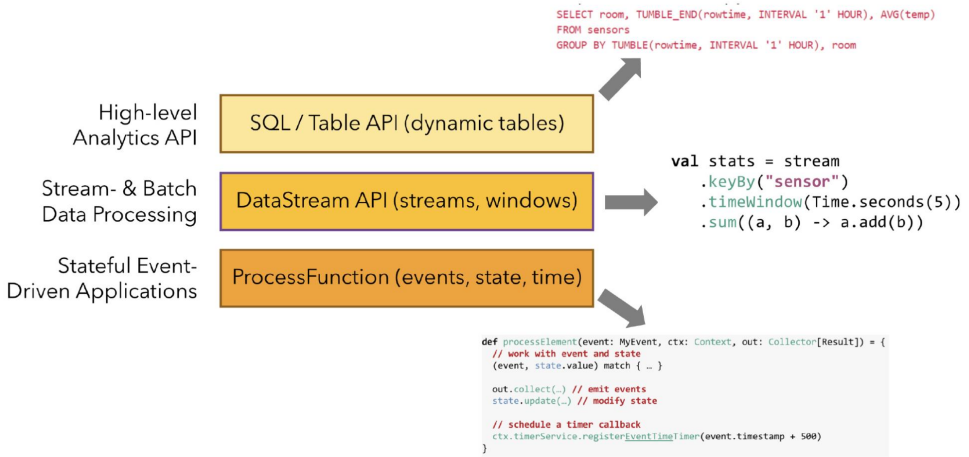
2.3 Flink架构

➢ 异常交易监测例子：

- Keyed State 维护每个用户的历史交易记录或传感器数据。
- Sliding Window / Tumbling Window 计算过去一段时间的数据模式。
- CEP（复杂事件处理） 组合多个事件检测异常行为（如连续 3 次大额交易）。

➢ 用户行为分析例子：

- Keyed State 存储用户的最近浏览/点击记录。
- RocksDB 状态后端 存储较大规模的用户行为数据，支持查询。
- ProcessFunction 处理自定义用户行为流，如用户连续点击某类商品后推荐相似商品。



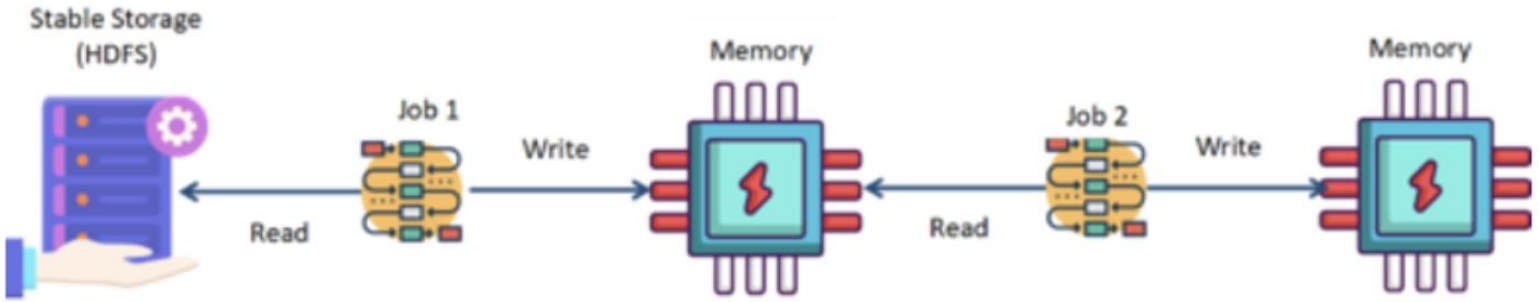
2.4 Spark Streaming架构

Spark

一种基于内存的分布式并行计算框架，不同于MapReduce的是——Job中间输出结果可以保存在内存中，从而不再需要读写HDFS，因此Spark能更好地适用于数据挖掘与机器学习等需要迭代的MapReduce的算法。

Spark RDD（Resilient Distributed Dataset）是 Spark 中最基本的数据抽象，它代表一个不可变、可分区、元素可以并行计算的数据集合。





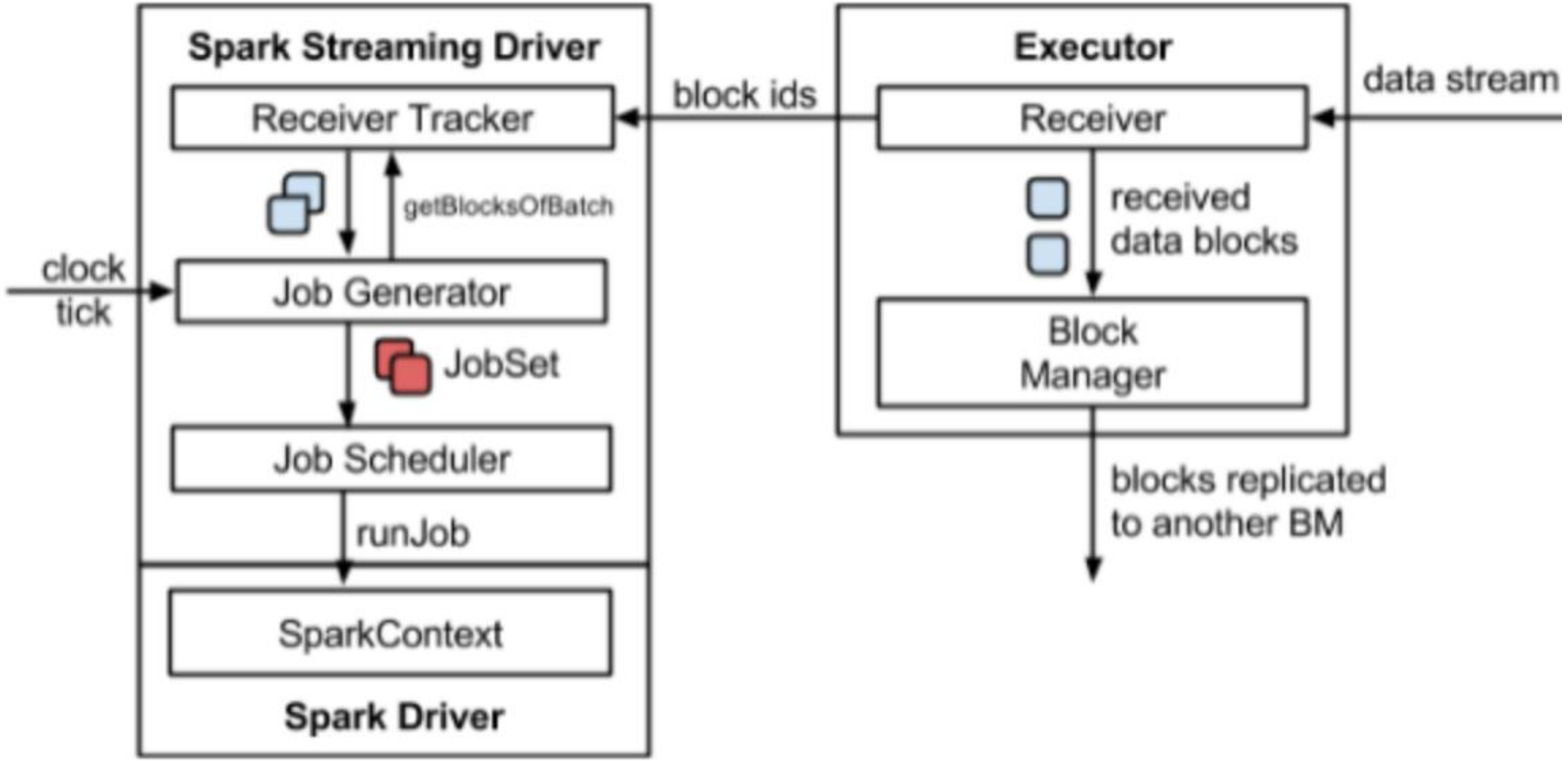
2.4 Spark Streaming架构

- Spark Streaming 是批处理的流式实时计算框架
  - 基于 Spark Core Api，因此其能够与 Spark 中的其他模块保持良好的兼容性，为编程提供了良好的可扩展性；
  - 粗粒度的准实时处理框架，一次读取完成,或异步读完数据之后，再处理数据，且其计算可基于大内存进行，因而具有较高的吞吐量；
  - 采用统一的 DAG 调度以及 RDD，对实时计算有很好的容错支持；



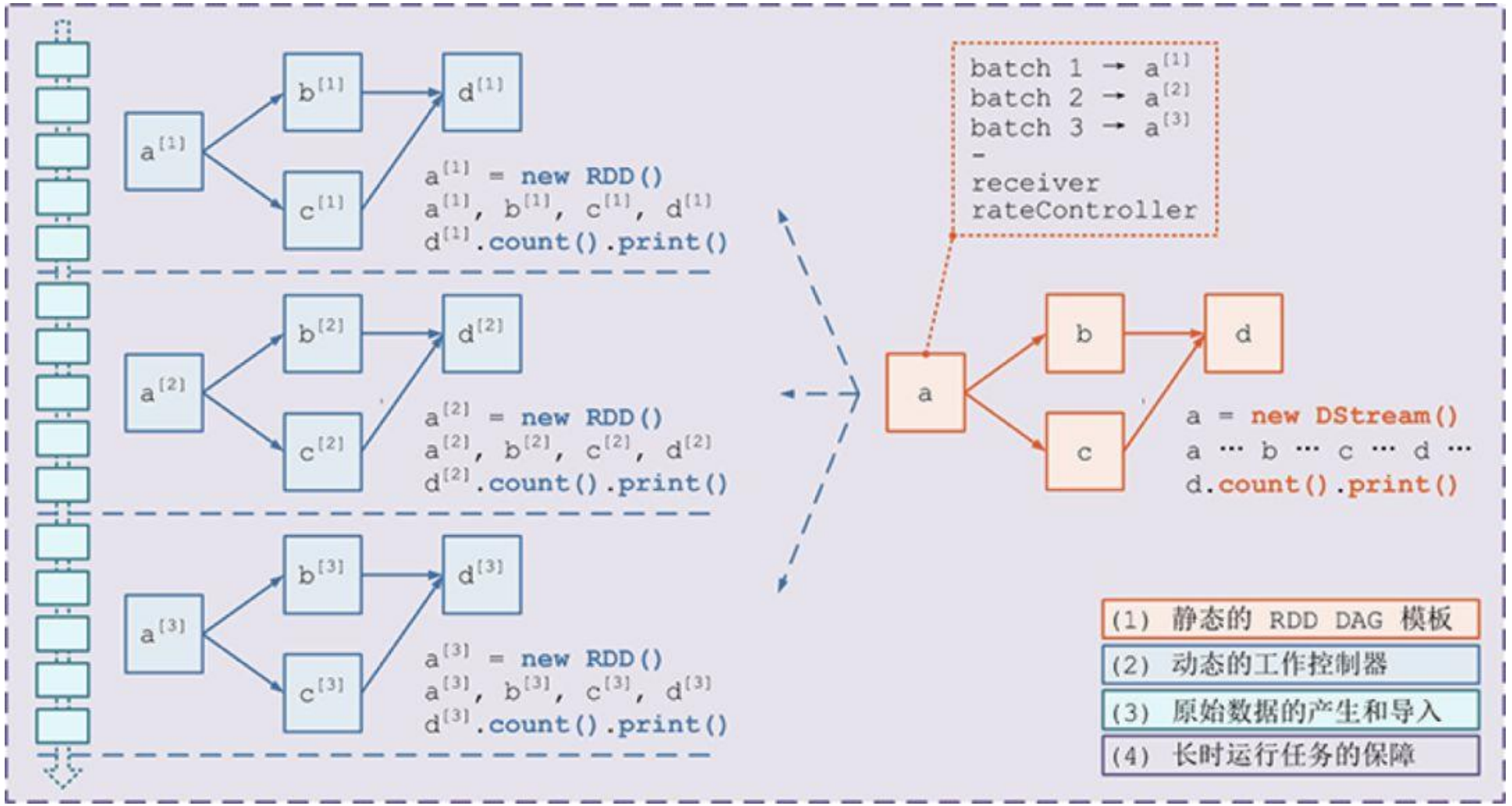
2.4 Spark Streaming架构

整体架构



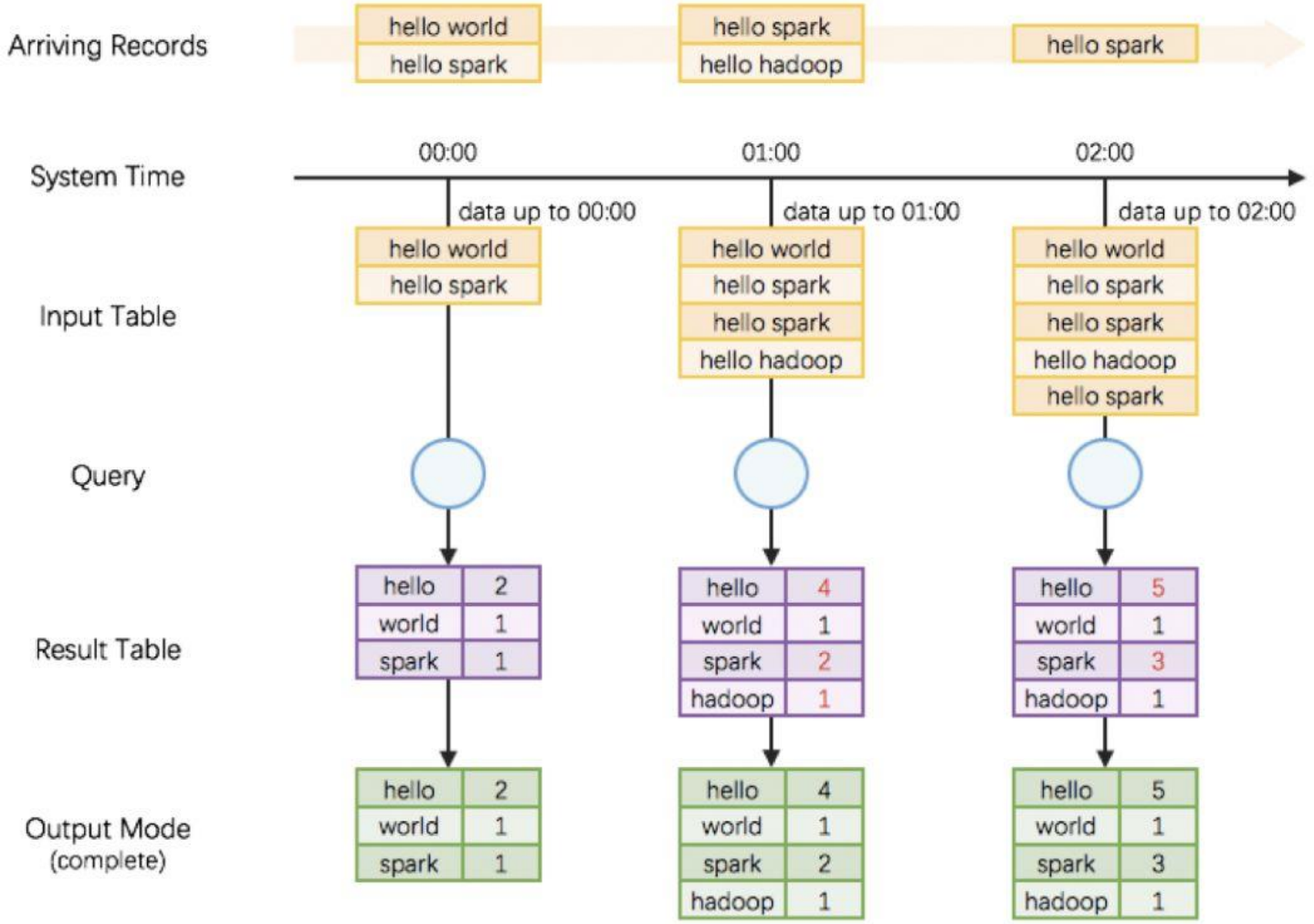
2.4 Spark Streaming架构

- 微批处理（micro-batch）



2.4 Spark Streaming架构

- 微批处理（micro-batch）



2.5 三种架构对比

维度	Storm	Flink	Spark Streaming
核心架构	基于 DAG（拓扑图），每个节点处理流数据	基于 Dataflow（流数据流模型）	基于 **微批处理（Micro-Batch）**
计算模式	**纯流处理**（每条数据独立处理）	**流+批一体**，原生流计算	**微批处理**（将数据按批次处理）
吞吐量	高（单条处理，低延迟）	极高（流+批优化，低延迟）	较高（批处理方式导致一定延迟）
延迟	ms **级**（超低）	ms **级**（超低）	**秒级**（微批机制，延迟 0.5s~2s）
状态管理	需要手动维护（如 Redis）	**内置状态管理（Keyed State）**	依赖 Checkpoint+HDFS
故障恢复	At-Least-Once（默认），需要手动保证 Exactly-Once	Exactly-Once（默认支持）	At-Least-Once/Exactly-Once**（配合 WAL）**

2.5 三种架构对比

维度	Storm	Flink	Spark Streaming
窗口计算	依赖 Trident（扩展库），功能有限	**强大支持**（基于 Event Time，支持滚动、滑动、会话窗口）	**基于时间的窗口**
流式 SQL	**无原生 SQL**（依赖 Trident）	**支持 Flink SQL**	**支持 Spark SQL**
生态集成	Kafka、HBase、Redis	Kafka、HDFS、Elasticsearch、Flink ML	Kafka、HDFS、MLlib
应用场景	**低延迟场景**（实时监控、告警）	**流+批统一**（推荐、风控、IoT）	**大数据 ETL、批流一体**
开发难度	**较高**（手写 Bolt/Spout）	**适中**（DataStream API 简单易用）	**较低**（接近批处理方式）

目录

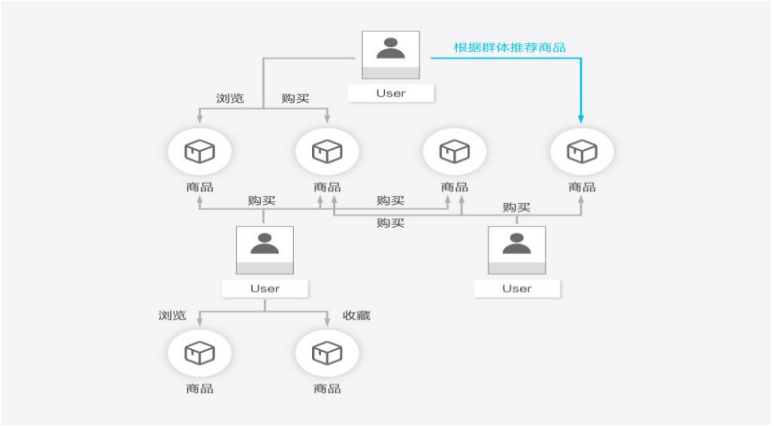
- 1 数据智能平台计算架构回顾
- 2 流计算模式设计
- 3 图计算模式设计
- 4 小结

3 图计算

- 许多大数据都是以**大规模图**或**网络**的形式呈现。从中挖掘出有价值的信息、知识和规律，为事件溯源、并分析因果关系等，可以为实际业务应用提供支持。
  - 如社交网络、传染病传播途径、交通事故对路网的影响
- 数据之间的关联关系过于复杂，**传统的关系型数据难以承载**，也基本无法实现高效率的计算，更无法挖掘数据间复杂关联关系里面的价值或因果关系。
- 而图数据由一系列的点、边构成，能更自然、直观地表述**数据间的关联关系**，是一种更符合人类思考方式的抽象表达。



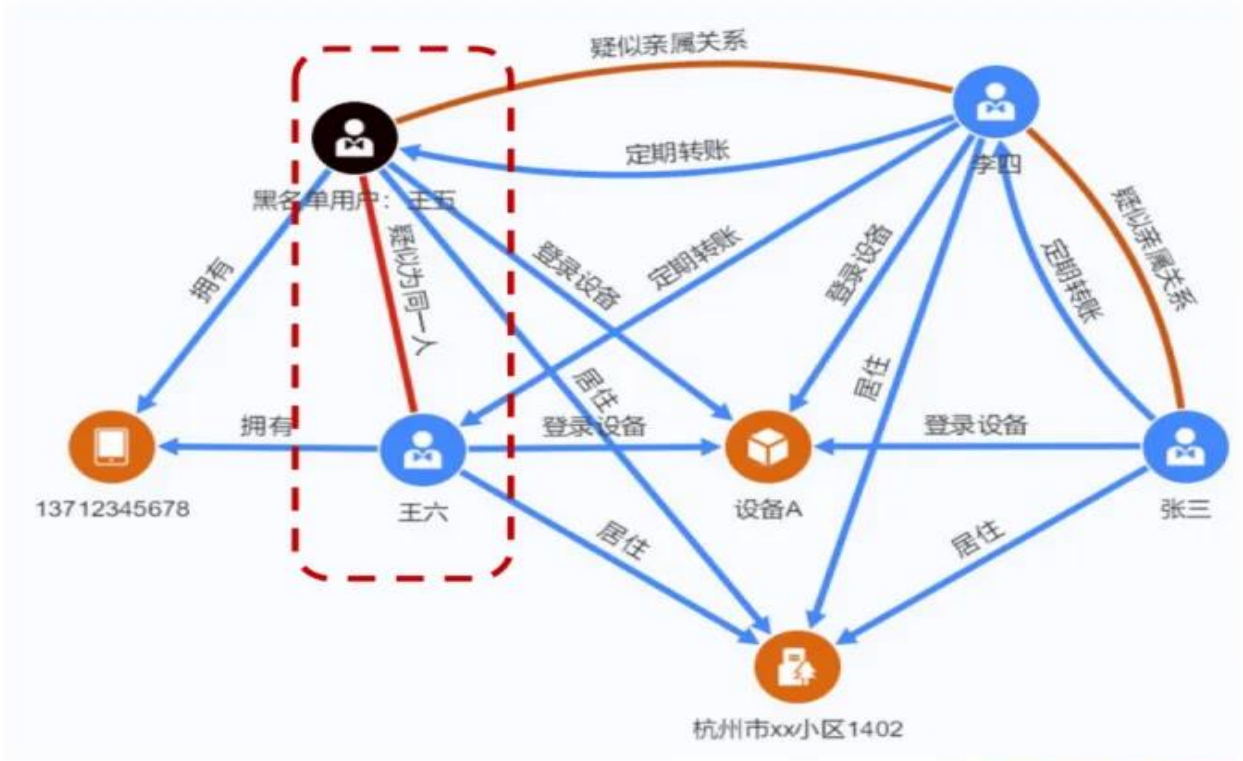
社交网络



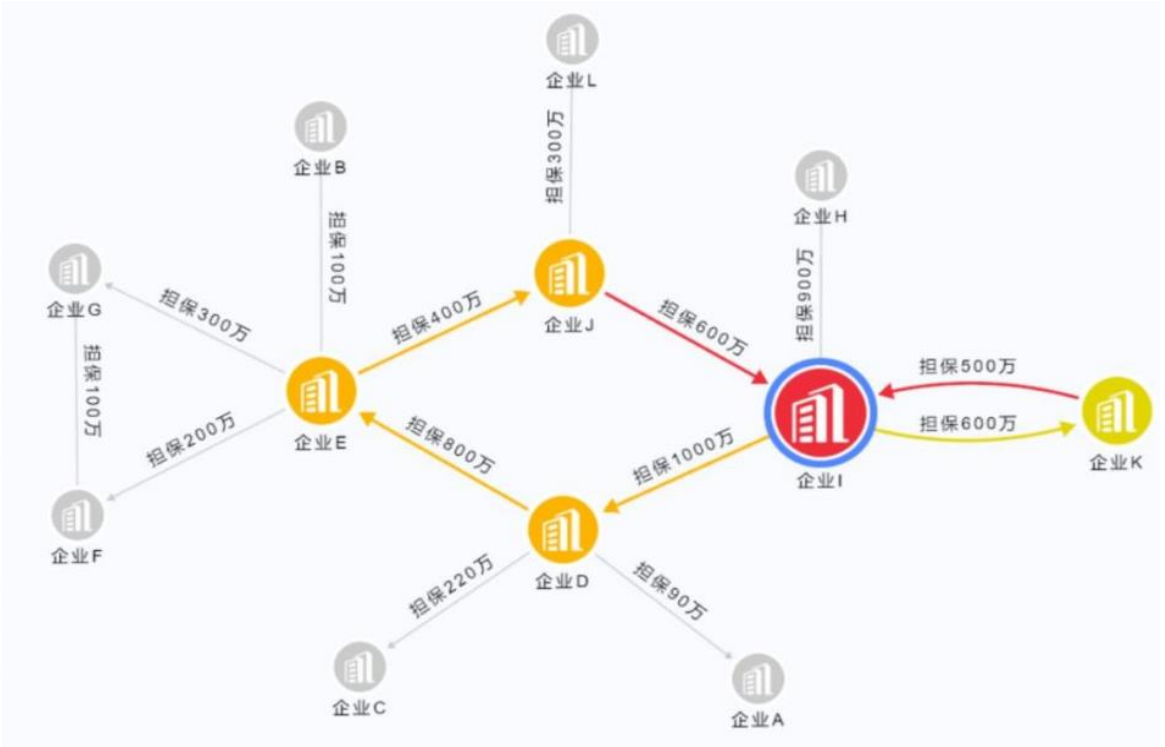
商品推荐

为什么需要图计算?

➢ 金融风控



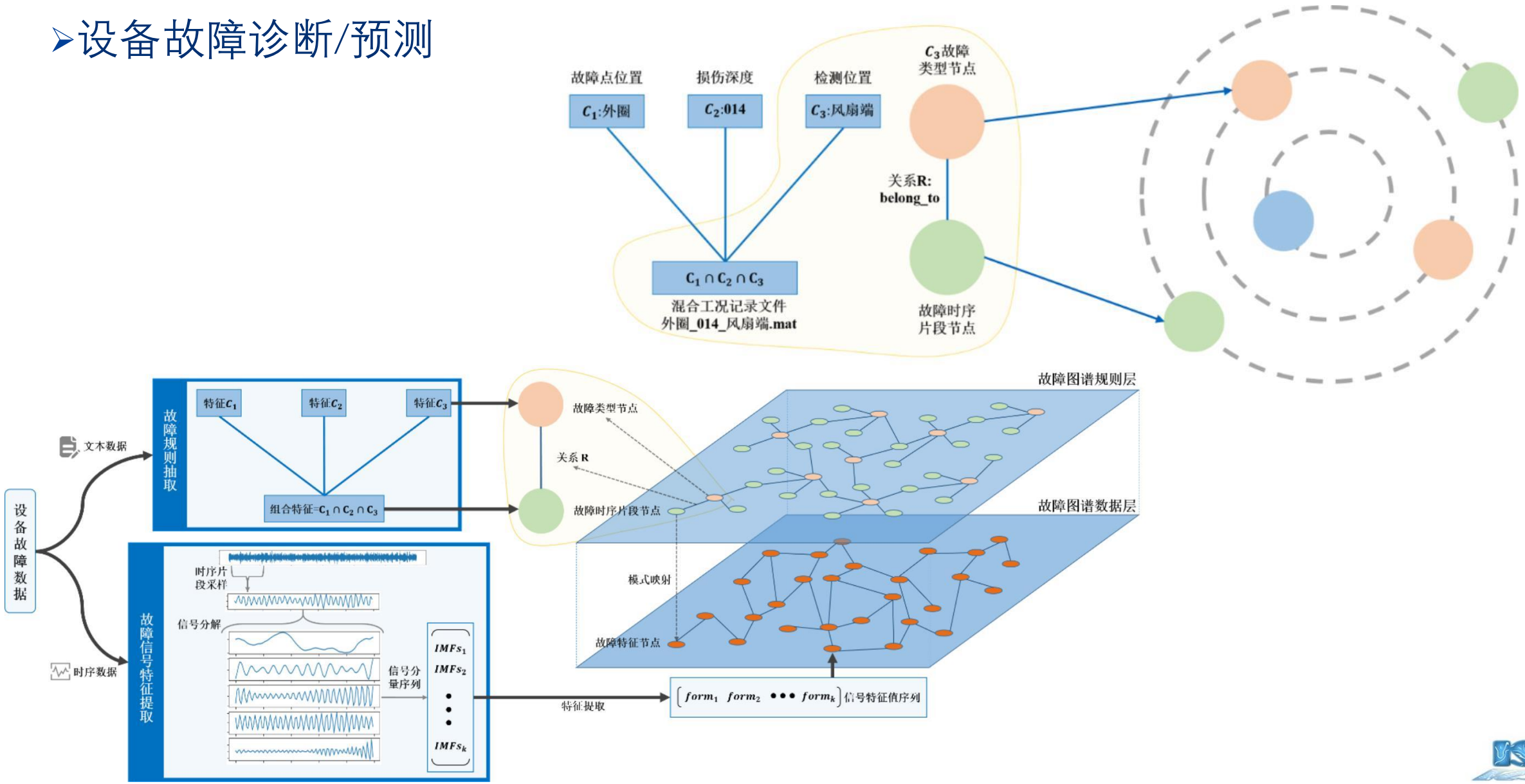
身份识别：通过使用图计算找共同邻居，分析两点间的相似度，判断不同节点是否为同一人



信贷风险检测：全图遍历，识别出特定的有向连通网络结构，检测出互保、连环担保等特殊担保形态，预警当前信贷申请。

为什么需要图计算?

- 设备故障诊断/预测



图计算概述

- 图计算是一种**处理图结构数据的计算方式**，它将图数据中的节点和边视为计算单元和数据传输通道，并通过挖掘这些节点和边之间的关联来实现各种应用。
- 关键难点：
  - 图数据模型
    - 图数据模型是指如何将图数据存储在底层存储中，并提供对图数据的访问接口。数据模型的设计应该考虑到图数据的特点，例如顶点和边的属性、度分布等等。
    - 用于存储图数据的存储系统，通常是分布式图数据库，如Neo4j、Dgraph、JanusGraph、HugeGraph等。
  - 图计算算法
    - 一些常见算法和技术包括**图遍历、图搜索、最短路径算法、图聚类、图嵌入**等。近年来，随着大规模图数据的快速增长和深度学习技术的兴起，**图神经网络**（Graph Neural Networks，GNN）也成为了图计算的一个热门研究方向，它可以通过对图数据进行端到端的学习来实现各种任务。

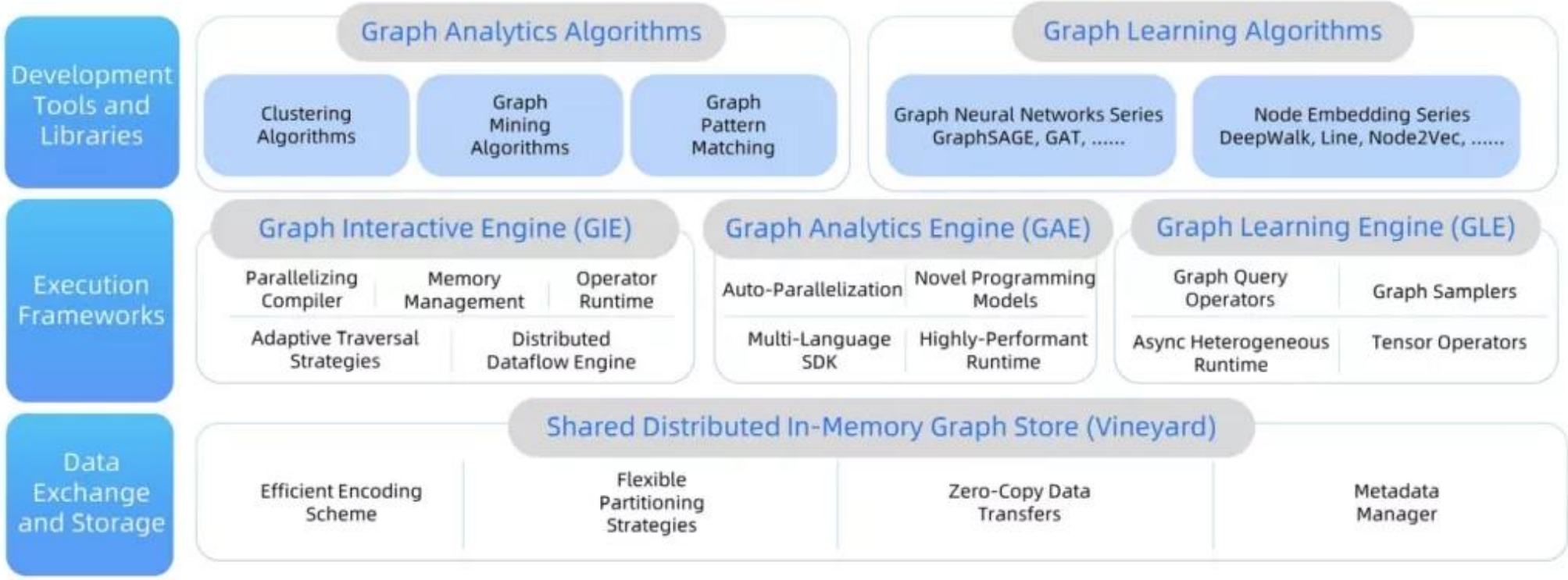
3.1 图计算架构

- 分布式图计算典型架构-以阿里巴巴GraphScope为例:

应用层

计算层

存储层



图计算典型架构

- 存储层：负责存储和管理大规模的图数据，提供数据读取、写入和查询等基本功能。
  - 常见的图数据存储包括分布式图数据库（如Neo4j、Dgraph、JanusGraph、HugeGraph）等。
  - 基于遍历算法的、实时的图计算架构
- 计算层：负责实现基于底层存储的图计算框架，包括图的构建、转换、分割和分布式计算等功能。
  - 常见的图计算框架包括Pregel、Giraph、GraphX、PowerGraph、GraphLab等。
  - 以图顶点为中心的、基于消息传递批处理的并行引擎
- 应用层：基于计算层提供的图计算框架，实现各种图算法。
  - 如PageRank、SSSP（单源最短路径）、LPA（标签传播）等，以及基于图数据的各种应用，如社交网络分析、推荐系统、网络安全等。

图计算特点

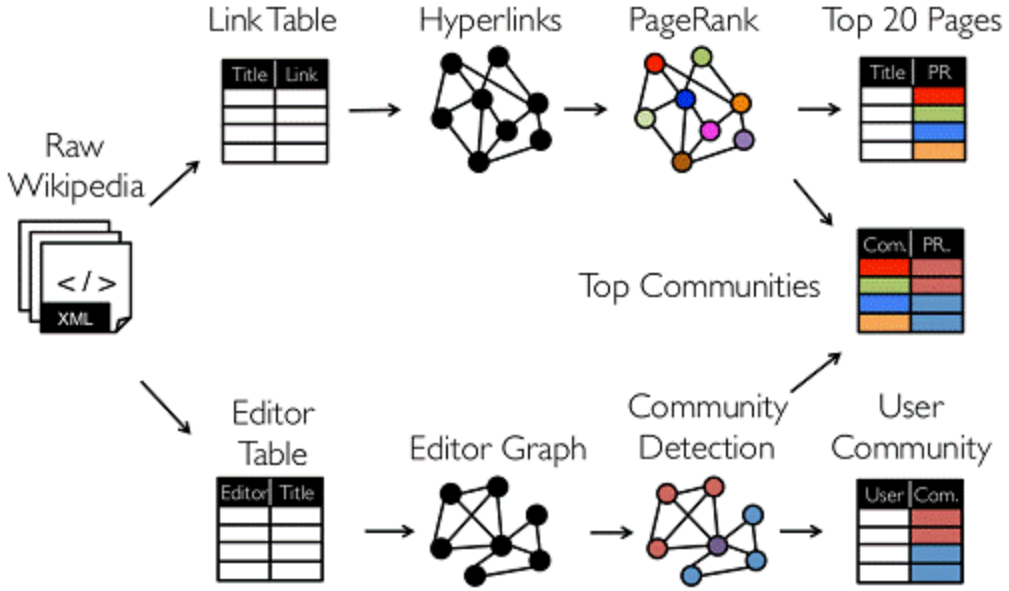
- 数据模型：
  - 将数据表示成**节点和边**，而不是传统的表格或矩阵形式，可以更好地描述实际世界中的复杂关系。
- 并行计算：
  - 图数据通常非常大且复杂，**无法通过单台计算机进行处理**。因此，图计算通常使用分布式计算框架来实现高效的并行计算。
- 迭代计算：
  - 图计算通常需要进行多轮迭代计算得到最终结果。**每一轮迭代计算都会更新图中节点的状态，直到收敛为止**。
- 局部计算：
  - 图计算通常基于局部计算模型，即**每个节点只考虑其直接相邻节点的状态**，并根据它们的状态进行计算。这种局部计算模型有助于降低计算复杂度，并提高计算效率。

图计算特点

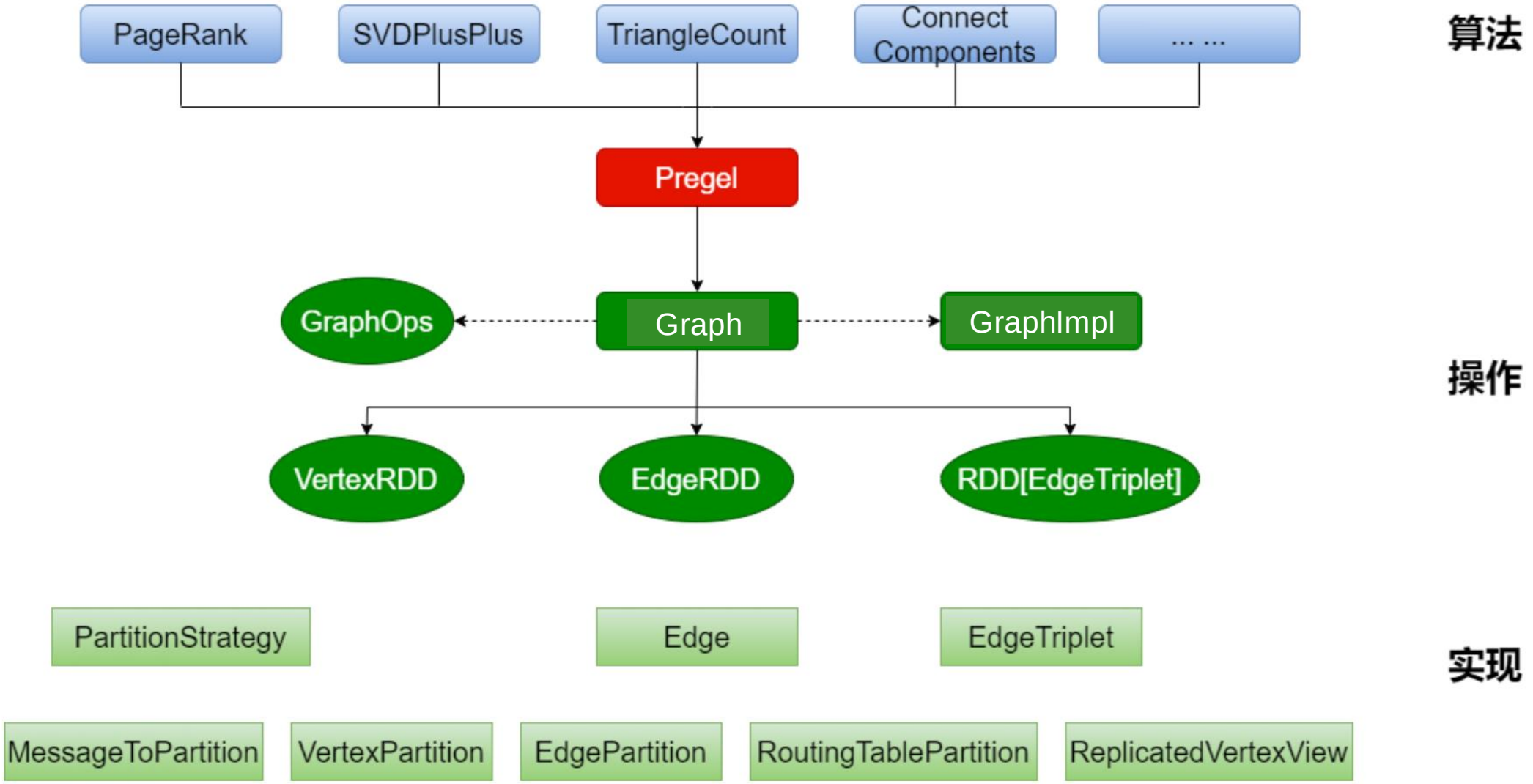
- 图计算的优点
  - 能够处理复杂的关系和结构：如节点之间的多重关系、环形结构等。
  - 能够处理大规模的图数据：图计算技术基于分布式计算框架。
  - 能够提供高效的并行计算能力：图计算技术基于分布式计算框架。
  - 能够应用于多种场景：如社交网络分析、推荐系统、生物信息学等。
- 图计算的缺点
  - 算法复杂度高：需要进行多轮迭代计算，每轮计算都需要更新节点状态，并进行局部计算。
  - 数据存储和管理难度大：图数据量通常非常大，且结构复杂，需要进行高效的存储和管理。
  - 缺乏标准化：不同的分布式计算框架和算法库之间缺乏统一的接口和标准，导致开发和使用难度较大。

3.2 Spark GraphX计算架构

- Spark GraphX是一个分布式图处理框架，它是基于Spark平台提供对图计算和图挖掘简洁易用的而丰富的接口，极大的方便了对分布式图处理的需求。
  - 图的分布式或者并行处理其实是把图拆分成很多的子图，然后分别对这些子图进行计算，计算的时候可以分别迭代进行分阶段的计算，即对图进行并行计算。



3.2 GraphX计算架构



3.2 GraphX计算架构

- Pregel

谷歌在后Hadoop时代的新“三驾马车”：Caffeine，Dremel，Pregel

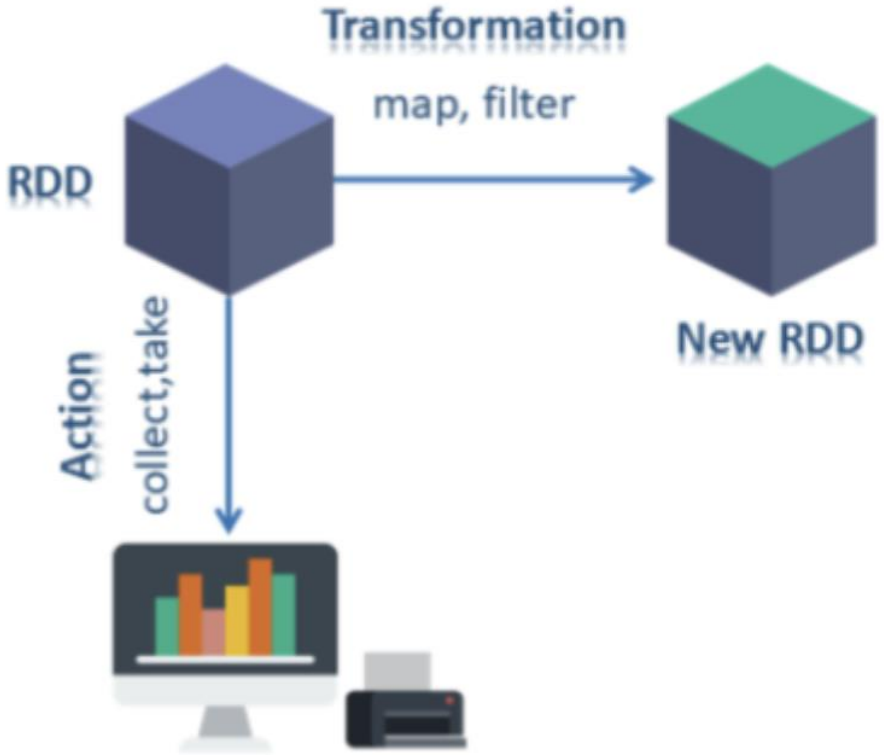
Pregel是一种基于BSP模型实现的并行图处理系统

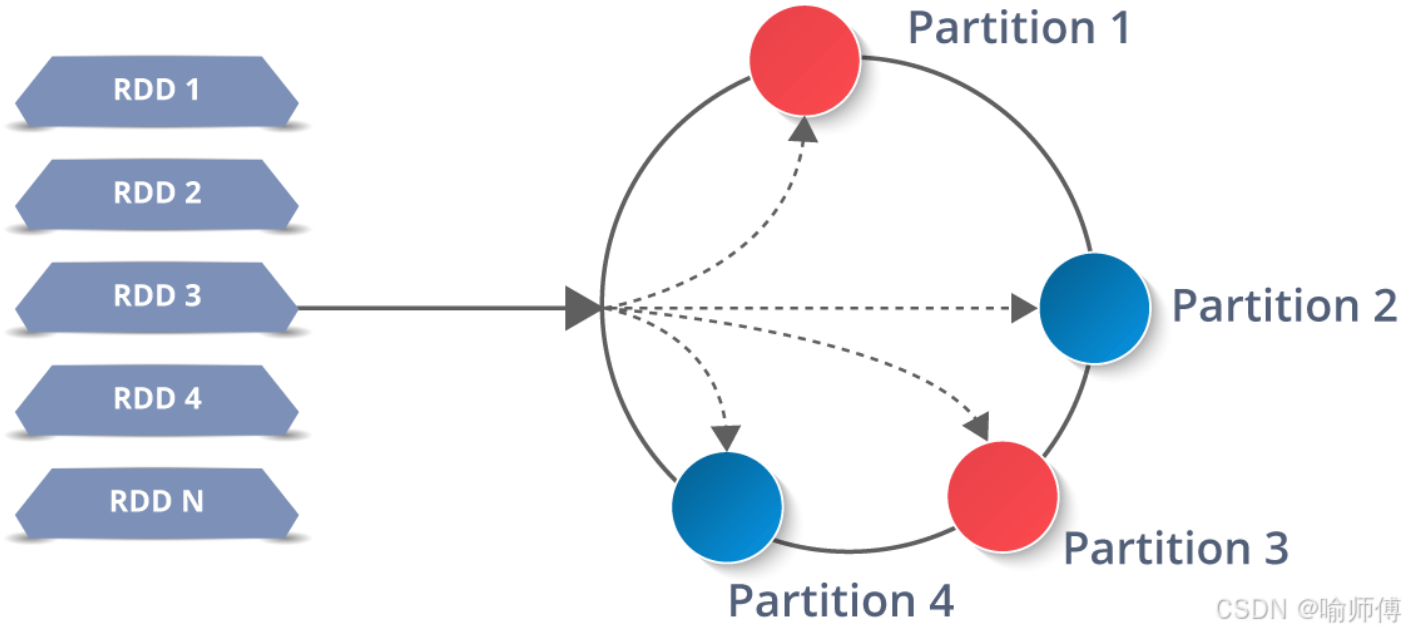
作为分布式图计算的计算框架，主要用于图遍历、最短路径、PageRank计算

3.2 GraphX计算架构

➢ 数据模型操作

- 两类算子（Transformation，Action）和分区（Partition）

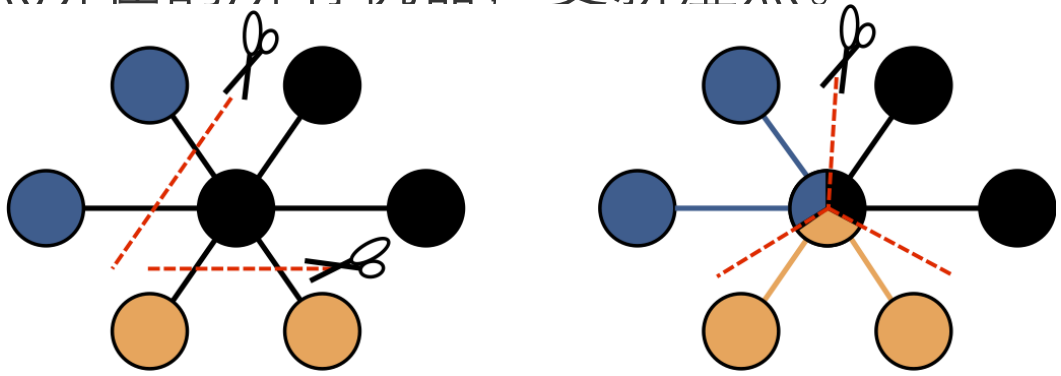




CSDN @喻师傅

3.2 GraphX计算架构

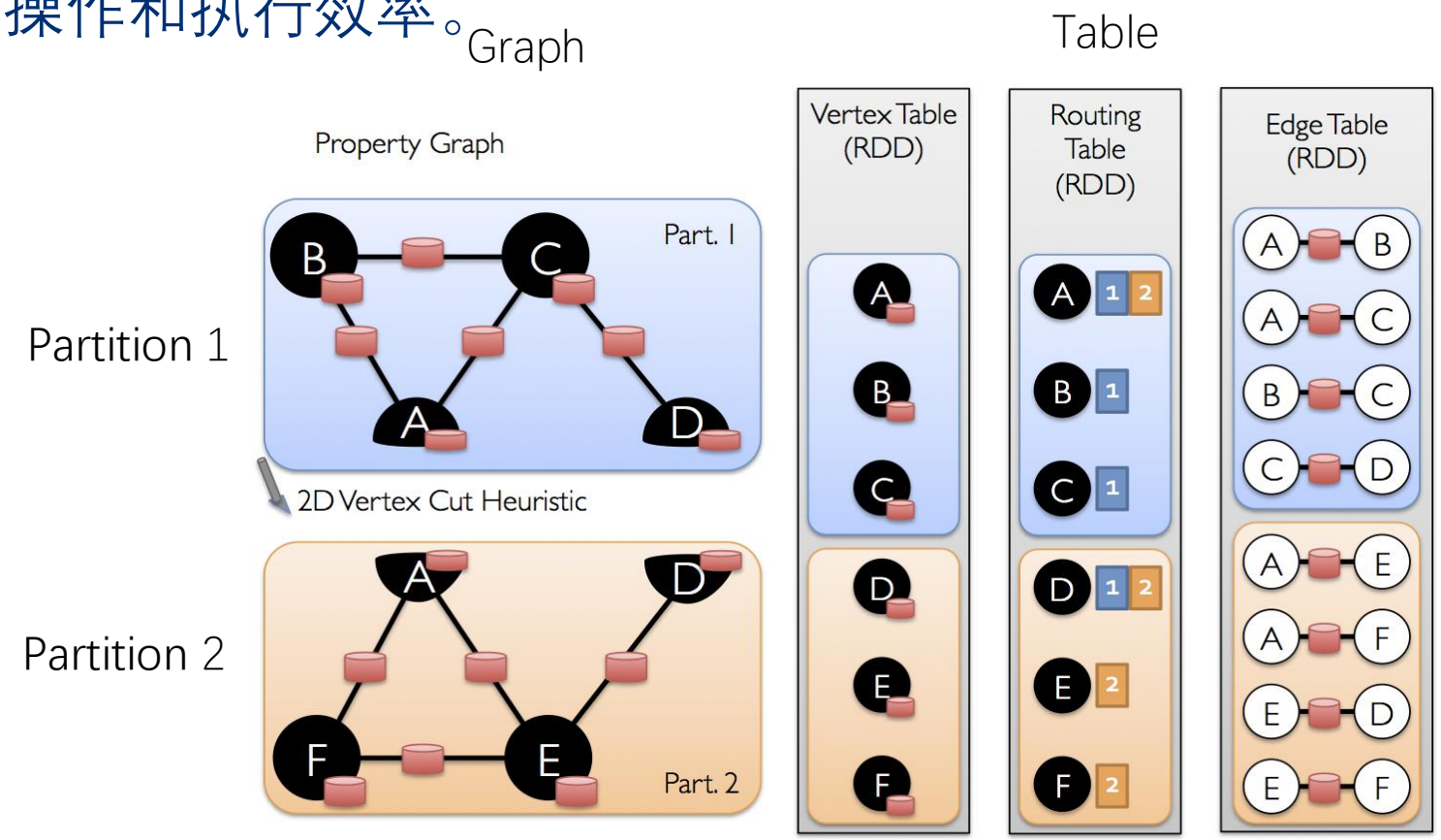
- 数据分区
- 图数据划分的两种方式：点分割存储和边分割存储。
- GraphX使用的是顶点分割（Vertex-Cut）方式存储图：任何一条边只会出现在一台机器上，每个点有可能分布到不同的机器上。
- 当点被分割到不同机器上时，是相同的镜像，但是有一个点作为主点,其他的点作为虚点，当点的数据发生变化时,先更新主点的数据，然后将所有更新好的数据发送到虚点所在的所有机器，更新虚点。



Edge Cut　　Vertex Cut

3.2 GraphX计算架构

- GraphX的数据模型是Resilient Distributed Property Graph，一种点和边都带属性的有向多重图。它扩展了Spark RDD的抽象，有Table和Graph两种视图，而只需要一份物理存储。两种视图都有自己独有的操作符，从而获得了灵活操作和执行效率。

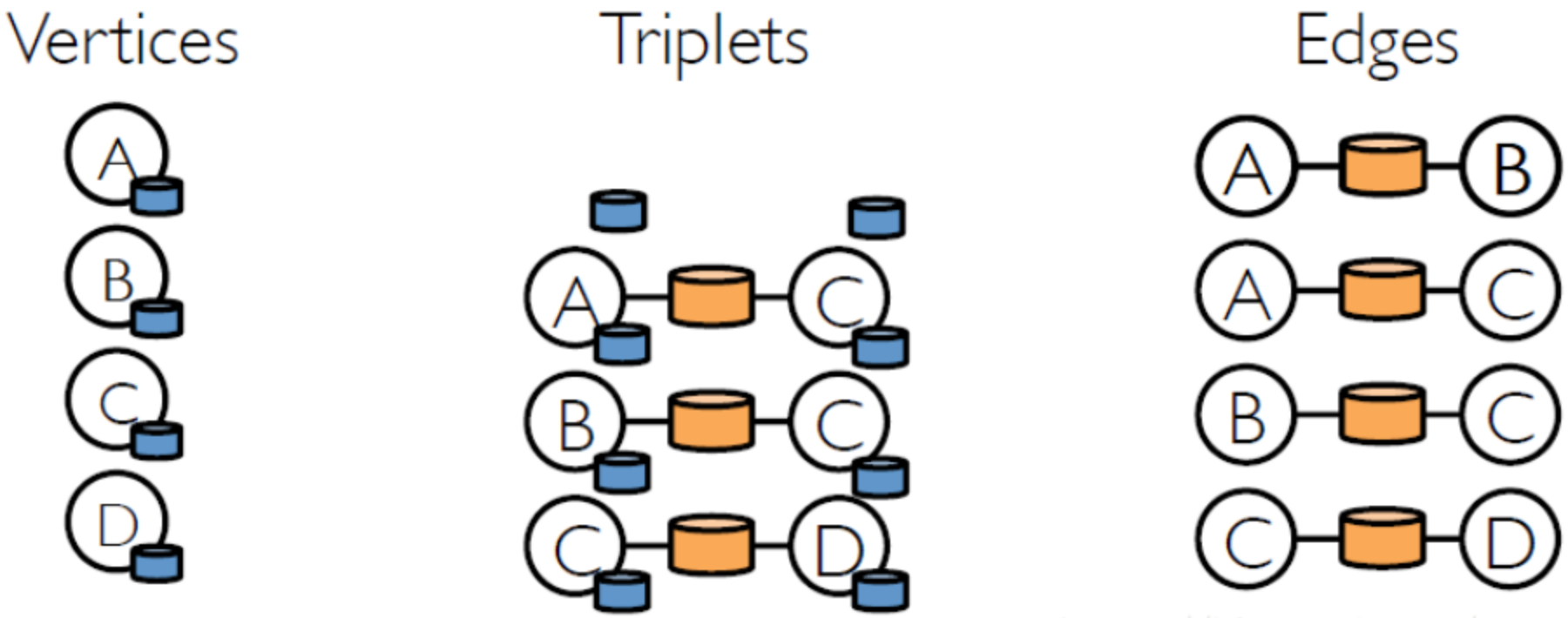


- Vertex Table(顶点Id,顶点属性)
- Edge Table(分区Id,源顶点id，目的顶点id, 边属性)
- Routing Table(点Id, 分区Id)
  - Vertex Table 和 Routing Table 的分区一一对应，Routing Table 表示一个vertex会涉及到哪些Edge Table partition。

3.2 GraphX计算架构

计算方式-迭代计算

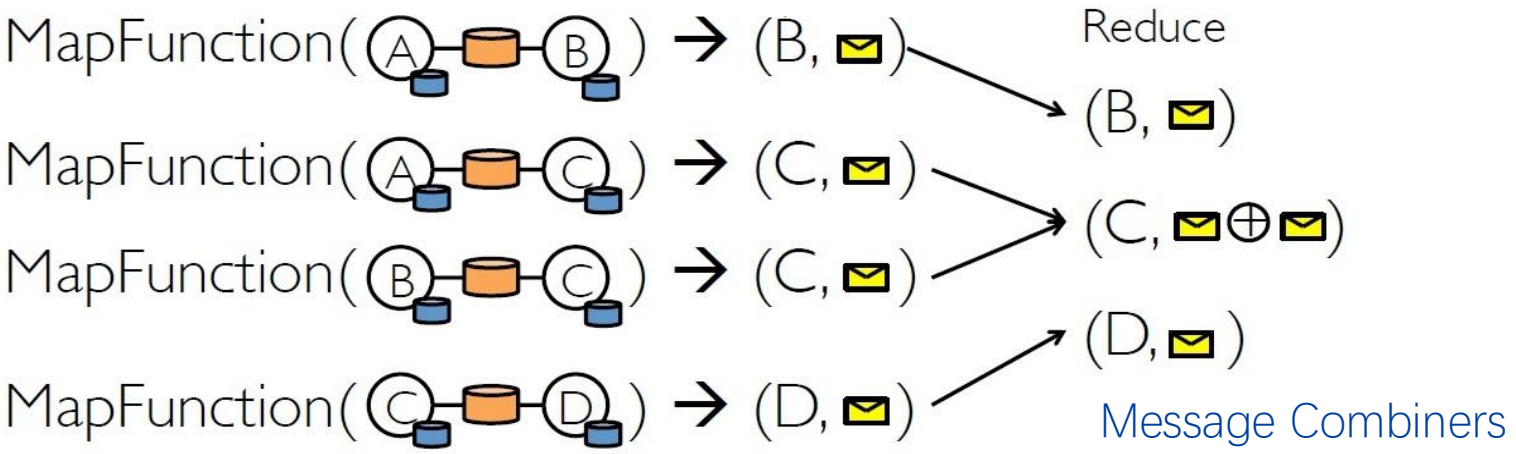
- 构造出一个Triplet (三元组视图)：通过将顶点RDD和边RDD进行Join操作。
  - Triplet包括源顶点属性、目标顶点属性、以及边属性。



3.2 GraphX计算架构

计算方式-迭代计算

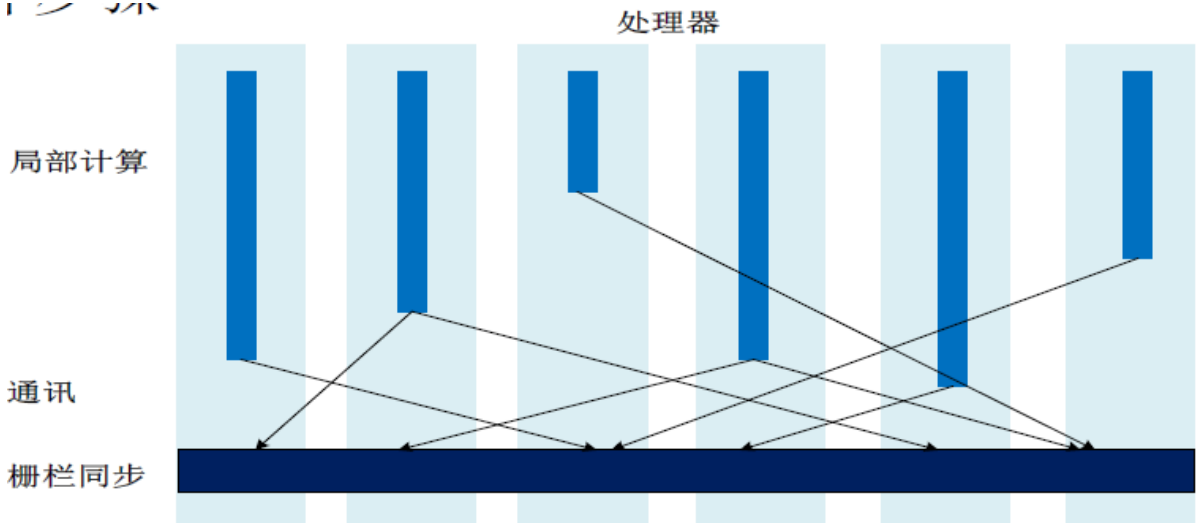
- 对每个Triplet的每个元素进行一个MapFunction操作，得到一个消息。
  - 这个消息包括属性数据以及一个目标顶点的ID。
- Message Combiners通过累加这些消息，然后将累加结构更新到相应顶点的属性数据。
- 重复这一过程，直到最终图形中每个顶点的属性数据收敛不再变化。



3.2 GraphX计算架构

计算方式-并行计算

- 采用BSP(Bulk Synchronous Parallel)模式，将计算分成一系列的超步的迭代
- 在每一个超步内**并行执行局部计算**与**同步**
  - 本地计算阶段，每个处理器只对存储在本地内存中的节点进行本地计算。
  - 全局通信阶段，每个节点将本地计算结果发送给邻居节点，邻居节点根据这些信息更新自己状态。
  - 栅栏同步阶段，当所有节点都完成本地计算和消息传递后，将所有节点的状态进行同步，然后进行下一轮计算。



3.3 Spark GraphX +Neo4j实例

Spark GraphX构建社群

- 准备数据
  - 构建点和边，然后调用GraphX接口生成图。

```
//构建用户节点
val users: RDD[(VertexId, String)] =
    spark.sparkContext.parallelize(Array((3L, "u3"), (7L, "u7"),(5L, "u5"), (2L, "u2"), (4L, "u4"),(6L, "u6"),(8L, "u8")))

//构建用户边
val relationships: RDD[Edge[String]] =
    spark.sparkContext.parallelize(Array(Edge(7L, 3L,""), Edge(5L, 3L,""),Edge(5L, 2L,""), Edge(6L, 4L,""),Edge(8L, 6L,"")))

//组合节点和边构建图
val graph = Graph(users, relationships)
```

3.3 Spark GraphX +Neo4j实例

Spark GraphX构建社群

- 构建社群
  - 调用图的接口直接获取连通图。

```
//从图中抽取出连通图
val components = graph.connectedComponents()

//获取连通图中的点，vertices是一个tuple类型，key分别为所有的顶点id，value为key所在的连通图id(连通图中顶点id最小值)
val vertices = components.vertices
```

查看社群构建结果

得到的vertices是一个tuple类型，key分别为所有的顶点id，value为key所在的连通图id(连通图中顶点id最小值):

vertices： (6,4) (8,4) (3,2) (7,2) (5,2)

3.3 Spark GraphX +Neo4j实例

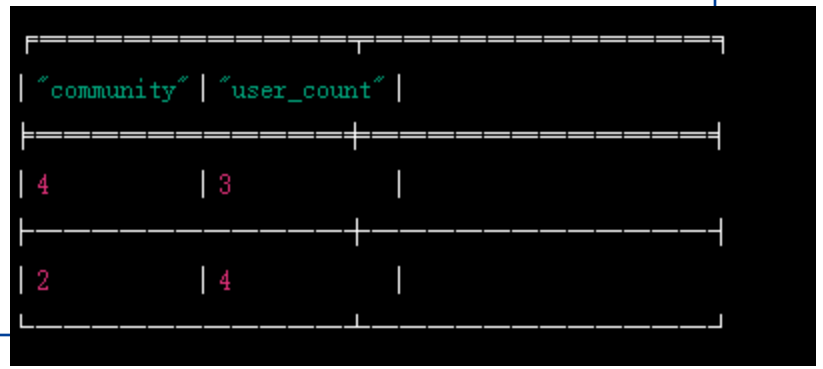
Neo4j存储社群

- 将社群存入图数据Neo4j以及查询

```
//链接neo4j
import org.neo4j.driver._
val driver = GraphDatabase.driver("bolt://localhost:7687", AuthTokens.basic("username", "password"))
val session = driver.session()

//存储: 创建一个名为“User”的节点标签，并为每个用户节点设置一个属性“id”和“community”。
vertices.foreach { case(vid, cid) =>
  val query = s"MERGE (u:User {id: $vid}) SET u.community = $cid"
  session.run(query)
}

//使用Cypher查询语言查询社区信息,这将返回每个社群的用户数
MATCH (u:User)
RETURN u.community, count(*)
```



"community"	"user_count"
4	3
2	4

3.4 Spark GraphX+PageRank实例

- PageRank算法
- 网页排名，又称网页级别、Google 左侧排名或佩奇排名。它是Google 创始人拉里· 佩奇和谢尔盖· 布林于1997 年构建早期的搜索系统原型时提出的链接分析算法。目前很多重要的链接分析算法都是在PageRank 算法基础上衍生出来的。PageRank 是Google 用于用来标识网页的等级/ 重要性的一种方法，是Google 用来衡量一个网站的好坏的唯一标准。在揉合了诸如Title 标识和Keywords 标识等所有其它因素之后， Google 通过PageRank 来调整结果，使那些更具“等级/ 重要性”的网页在搜索结果中令网站排名获得提升，从而提高搜索结果的相关性和质量。

3.4 Spark GraphX+PageRank实例

PageRank算法

这是Google最核心的算法，用于给每个网页价值评分，是Google“在垃圾中找黄金”的关键算法，这个算法成就了今天的Google

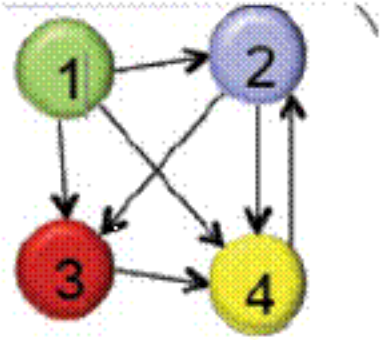
PageRank vector **q** is defined as $q = Gq$

where $G = \alpha S + (1-\alpha)\frac{1}{n}U$

- S is the destination-by-source stochastic matrix,
- U is all one matrix.
- n is the number of nodes
- α is the weight between 0 and 1 (e.g., 0.85)

Algorithm: Iterative powering for finding the first eigen-vector

$$q^{next} = Gq^{cur}$$



$$G = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1/3 & 0 & 0 & 1 \\ 1/3 & 1/2 & 0 & 0 \\ 1/3 & 1/2 & 1 & 0 \end{bmatrix}$$

3.4 Spark GraphX+PageRank实例

测试数据

测试数据为顶点数据graphx-wiki-vertices.txt和边数据graphx-wiki-edges.txt

```
138  3932908833397101503 St. Michael's College School
139  27084185981172377725 Metropolitan Junior Hockey League
140  50909562821672332l9 List of mountain ranges of California
141  41022239890966467 79 Java (programming language)
```

```
 98  17465170893509762 81 4439528526376405 03
 99  17465170893509762 81 4970488197231436 76
100  17465170893509762 81 5335442758331687 61
```

```
//设置运行环境
  val conf = new SparkConf().setAppName("PageRank").setMaster("local")
  val sc = new SparkContext(conf)

//读入数据文件
  val articles: RDD[String] = sc.textFile("/home/hadoop/IdeaProjects/data/graphx/graphx-wiki-vertices.txt")
  val links: RDD[String] = sc.textFile("/home/hadoop/IdeaProjects/data/graphx/graphx-wiki-edges.txt")
```

3.4 Spark GraphX+PageRank实例

算法运行

```
//装载顶点和边
  val vertices = articles.map { line =>
    val fields = line.split('\t')
    (fields(0).toLong, fields(1))
  }

  val edges = links.map { line =>
    val fields = line.split('\t')
    Edge(fields(0).toLong, fields(1).toLong, 0)
  }
//cache操作
  //val graph = Graph(vertices, edges, "").persist(StorageLevel.MEMORY_ONLY_SER)
  val graph = Graph(vertices, edges, "").persist()

//pageRank算法里面的时候使用了cache()，故前面persist的时候只能使用MEMORY_ONLY
 val prGraph = graph.pageRank(0.001).cache()
```

3.4 Spark GraphX+PageRank实例

➢ 结果输出

```
val titleAndPrGraph = graph.outerJoinVertices(prGraph.vertices) {
    (v, title, rank) => (rank.getOrElse(0.0), title)
  }

titleAndPrGraph.vertices.top(10) {
    Ordering.by((entry: (VertexId, (Double, String))) => entry._2._1)
  }.foreach(t => println(t._2._2 + ": " + t._2._1))
sc.stop()
```

PageRank计算，获取最有价值的数据

****

University of California, Berkeley: 1321.111754312097

Berkeley, California: 664.8841977233583

Uc berkeley: 162.50132743397873

Berkeley Software Distribution: 90.4786038848606

Lawrence Berkeley National Laboratory: 81.90404939641944

George Berkeley: 81.85226118457985

Busby Berkeley: 47.871998218019655

Berkeley Hills: 44.76406979519754

Xander Berkeley: 30.324075347288037

Berkeley County, South Carolina: 28.908336483710308

目录

- 1 数据智能平台计算架构回顾
- 2 流计算模式设计
- 3 图计算模式设计
- 4 小结

4 小结

- 流计算是实时获取来自不同数据源的海量数据，经过实时分析处理，获得有价值的信息的新型计算模式，广泛应用在实时智能应用场景。
  - Storm，Flink，Spark streaming三种典型流计算架构
  - Storm低时延，Flink高可靠，Spark高并发
- 图计算是一种处理图结构数据的计算方式，包含图存储、图计算、图算法，多应用在异构复杂关系场景、知识化场景。
  - 以GraphX为例阐释了分布式图计算架构
  - 未来在新型计算架构方面，信创产品越来越多。